



- 加法器设计

- 乘法器设计

第十章

数据通路设计

加法器设计

二进制加法器

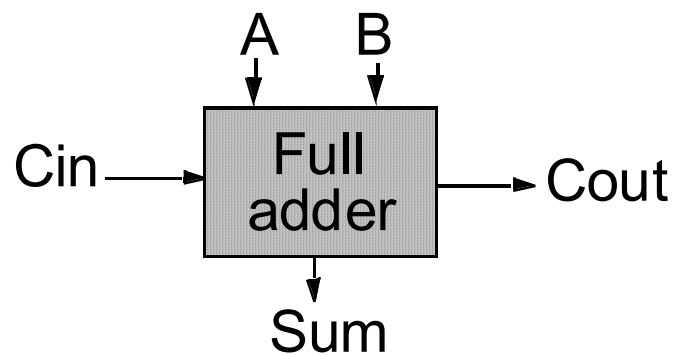
 改进串行进位加法器

 进位选择加法器

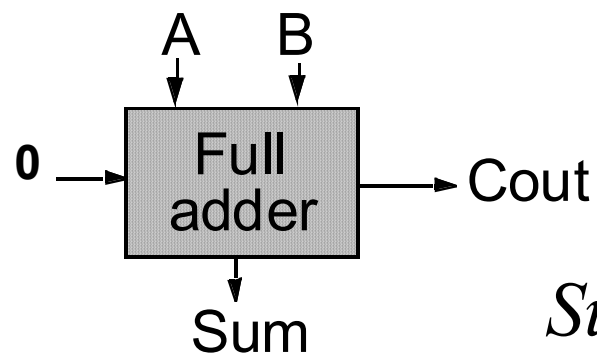
 超前进位加法器

 树形加法器

二进制加法



全加器



半加器

A	B	C_i	S	C_o	<i>Carry status</i>
0	0	0	0	0	delete
0	0	1	1	0	delete
0	1	0	1	0	propagate
0	1	1	0	1	propagate
1	0	0	1	0	propagate
1	0	1	0	1	propagate
1	1	0	0	1	generate
1	1	1	1	1	generate

$$Sum = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + C_{in}(A + B) \quad (\text{major运算})$$

进位产生、进位取消、进位传播信号

进位产生信号 (G)

$$G = AB$$

进位消除信号 (K或D)

$$K = \overline{AB}$$

进位传播信号 (P)

$$P = A \oplus B$$

统称情形 (setup)
信号

$$S(G, P) = P \oplus C_i$$

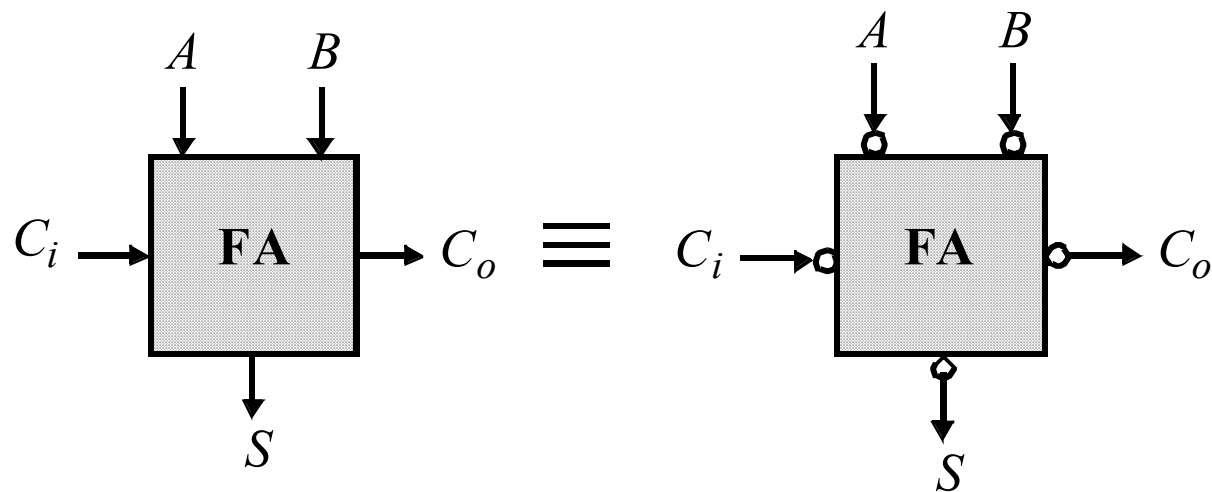
$$C_o(G, P) = G + PC_i$$

也可以取 $A+B$

$$C_o = AB + BC_i + AC_i$$

$$S = ABC_i + \overline{C_o}(A + B + C_i)$$

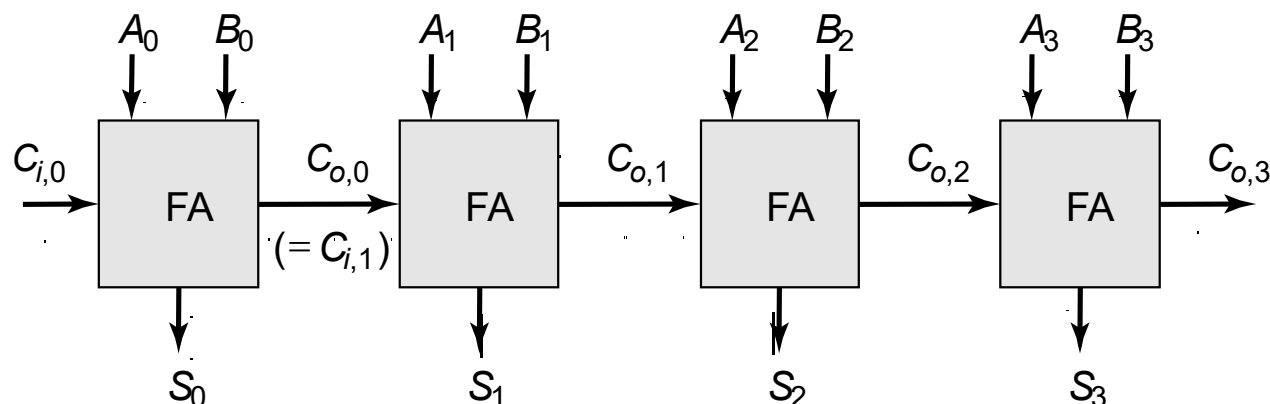
反相特性



$$\bar{S}(A, B, C_i) = S(\bar{A}, \bar{B}, \bar{C}_i)$$
$$\bar{C}_o(A, B, C_i) = C_o(\bar{A}, \bar{B}, \bar{C}_i)$$

N位加法器

行波进位（Ripple-Carry）加法器



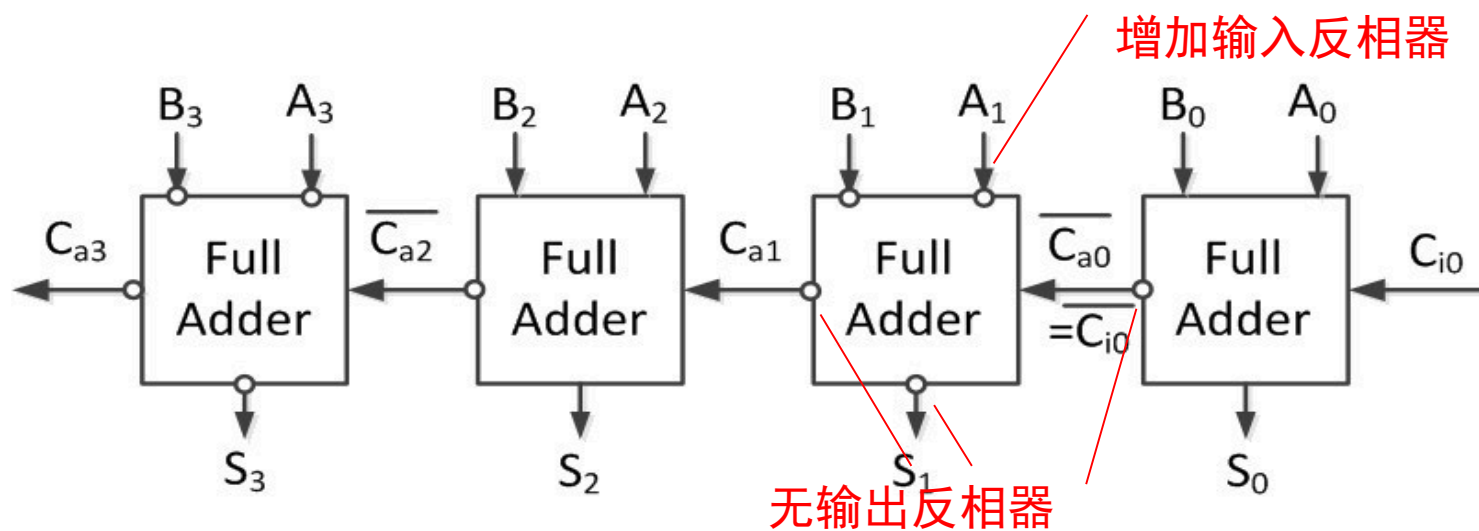
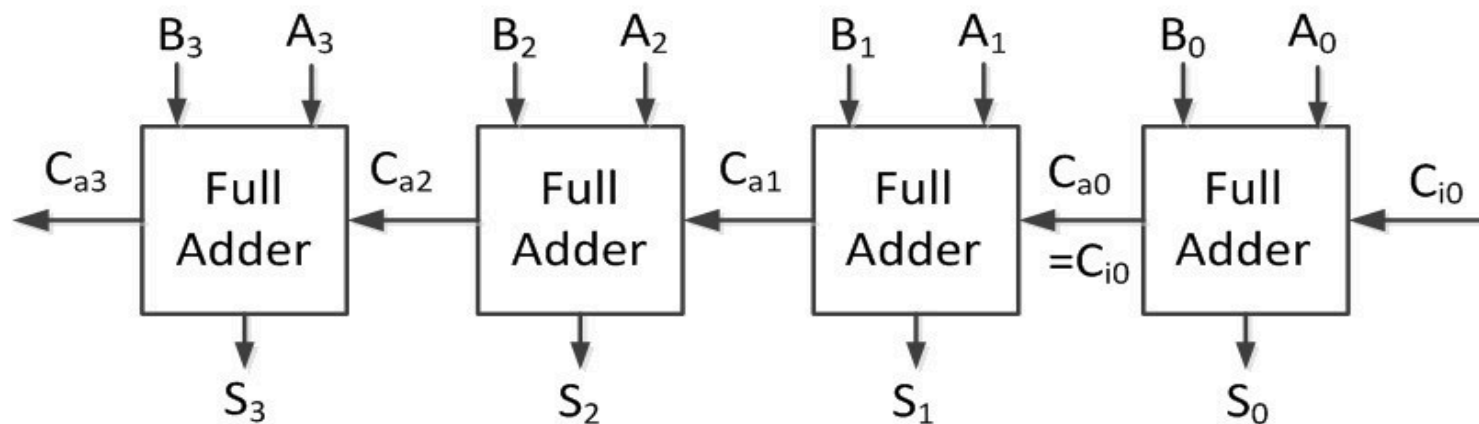
最坏情况的延时与加数位数 N 成线性关系

$$t_d = O(N)$$

$$t_{adder} = (N-1)t_{carry} + t_{sum}$$

关键：加快进位信号传播的速度

消除反相器的行波进位加法器



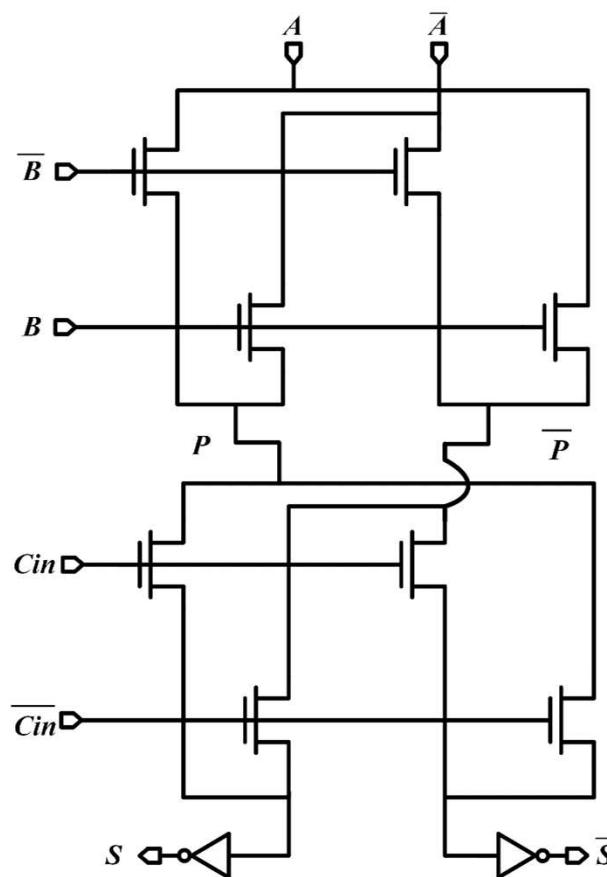
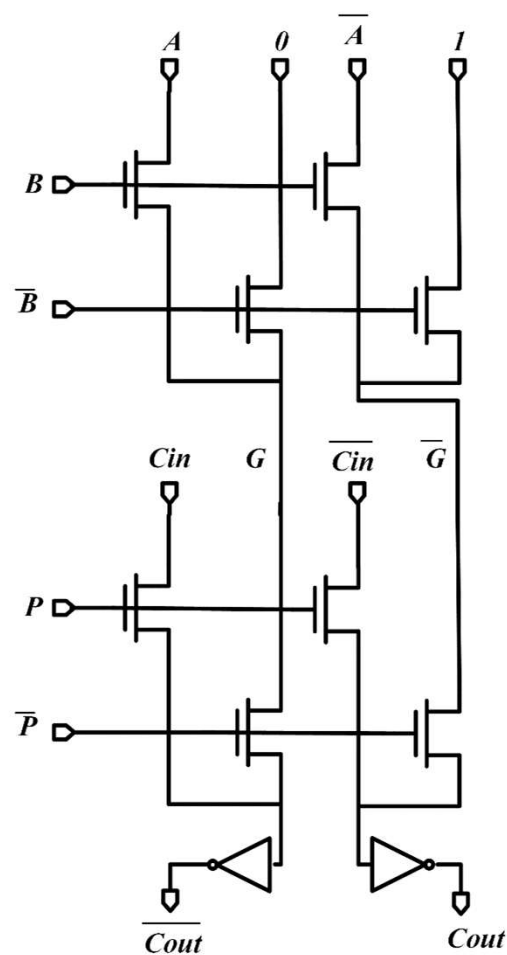
全加器电路（1）——CPL全加器

$$G = AB$$

$$P = A \oplus B$$

$$S = P \oplus C_{in}$$

$$C_{out} = G + PC_{in}$$

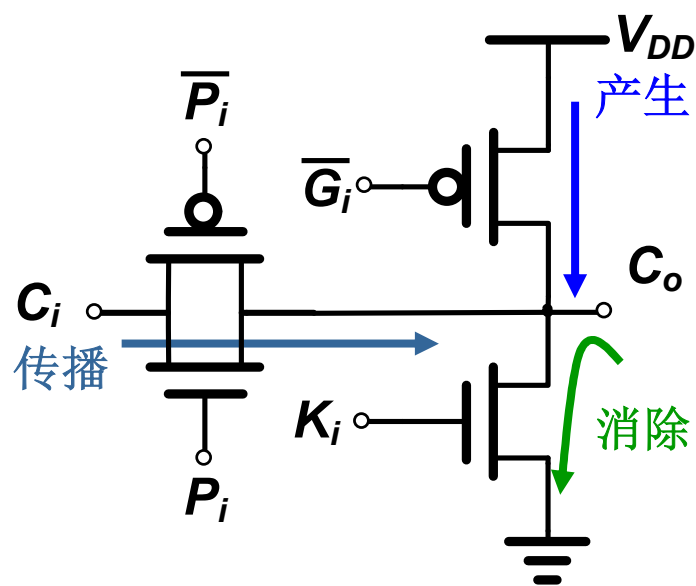


24个晶体管

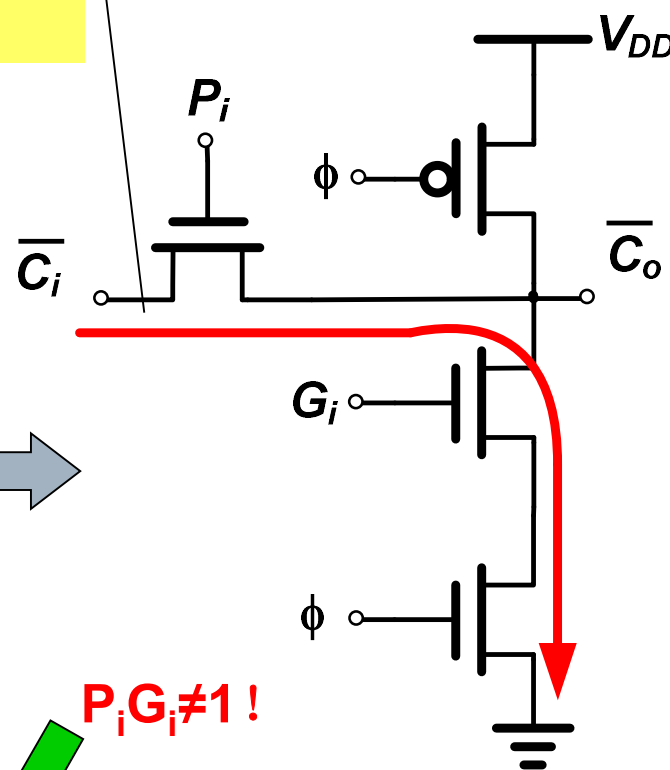
全加器电路（2）——曼彻斯特进位门

曼彻斯特（Manchester）进位门

若 $P_i=1$ 且
 $G_i=1$



静态电路

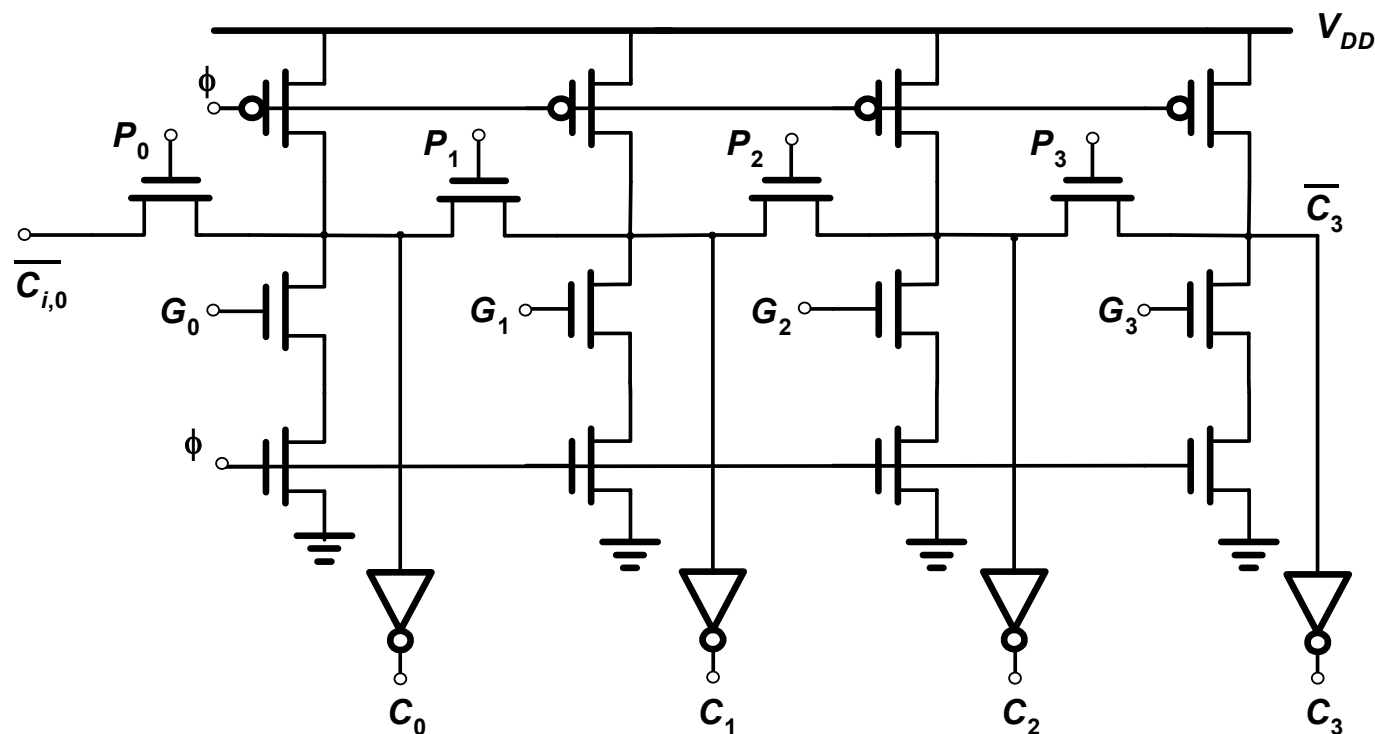


动态电路

$P_i G_i \neq 1!$

$$P_i = a_i \oplus b_i, G_i = a_i b_i$$

曼彻斯特 (Manchester) 进位链



- 采用动态逻辑降低复杂性和加快速度。
- 进位链传输管只用 N 管，节点电容很小，为四个扩散电容。

加法器设计

❖ 二进制加法器

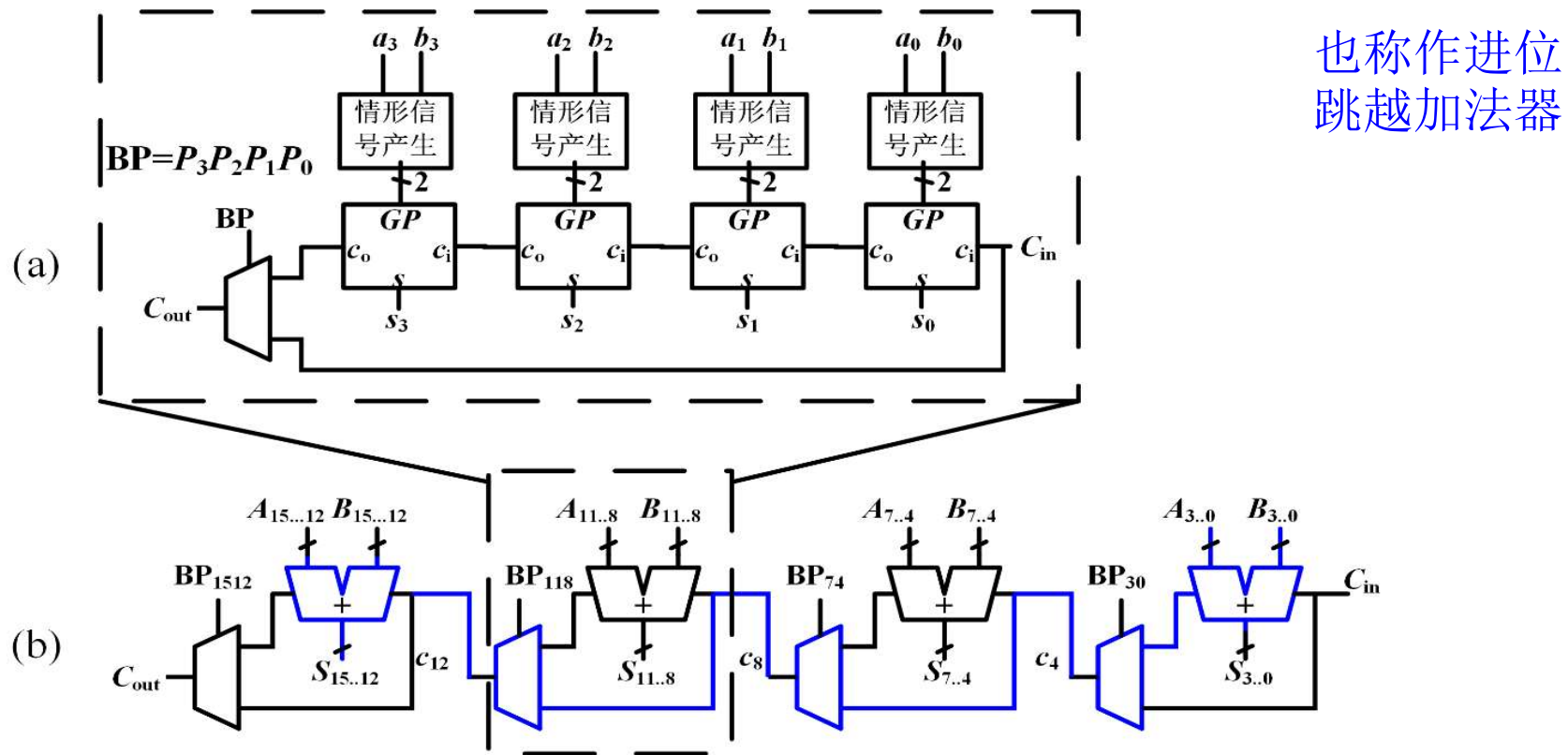
❖ 改进串行进位加法器

❖ 进位选择加法器

❖ 超前进位加法器

❖ 树形加法器

进位旁路(Carry-bypass)加法器

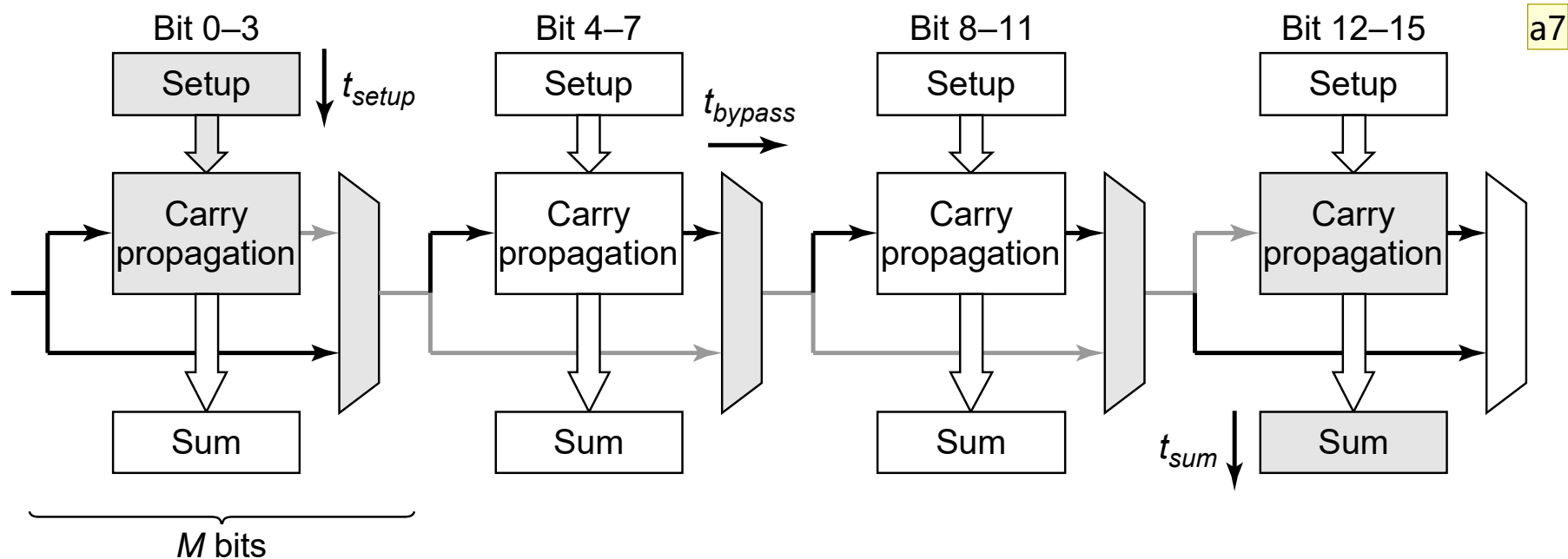


原理: 如果 (P_0 、 P_1 、 P_2 和 P_3 均=1), 则 $C_{O,3} = C_{i,0}$
否则主路径进位消除或进位产生。

进位旁路加法器的关键路径

N 位串行加法器分成组，每组 M 位

(Setup:产生G, P, K信号)



$$t_{adder} = t_{setup} + Mt_{carry} + (N/M-1)t_{bypass} + (M-1)t_{carry} + t_{sum}$$

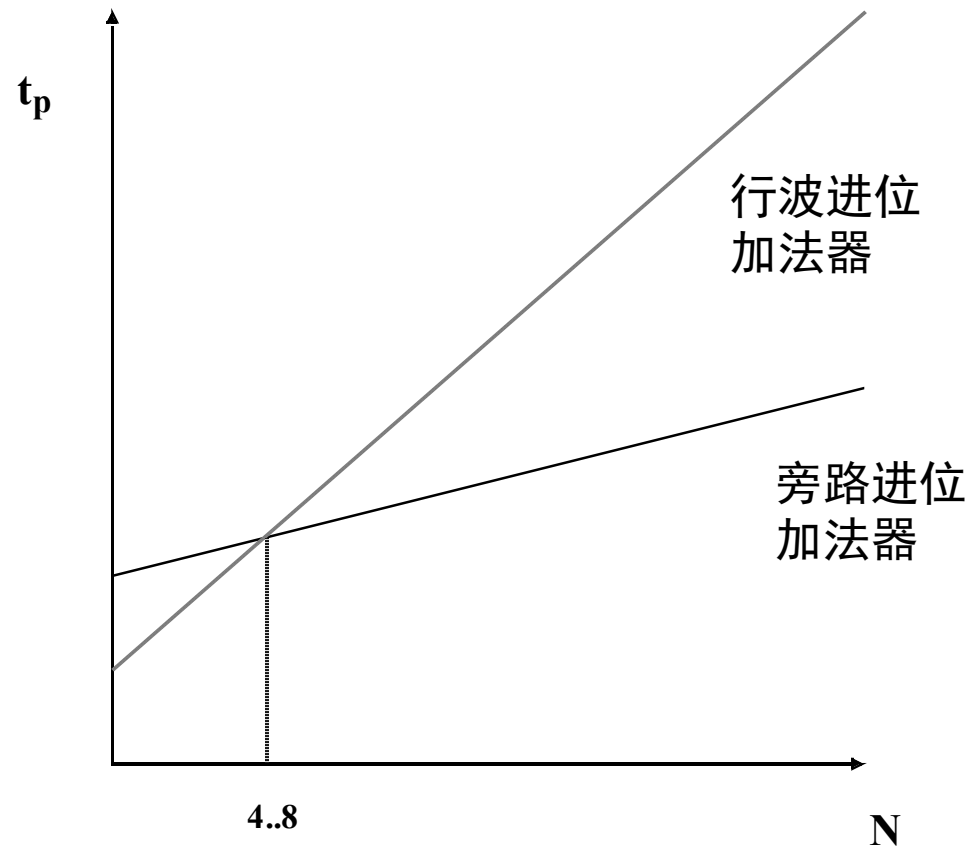
a7

如果第7位的进位输出（C7）与第3位的进位输出（C3）无关，那么输入到C7的关键路径就等于Bit4-7的进位链，这段计算是与Bit0-3的进位传输并行的；如果C7与C3无关，则必然满足bypass条件，则C3-C7的关键路径就是bypass路径，因而中间组的最坏情况是通过bypass路径。

adm, 2011/12/8

行波进位与进位旁路比较

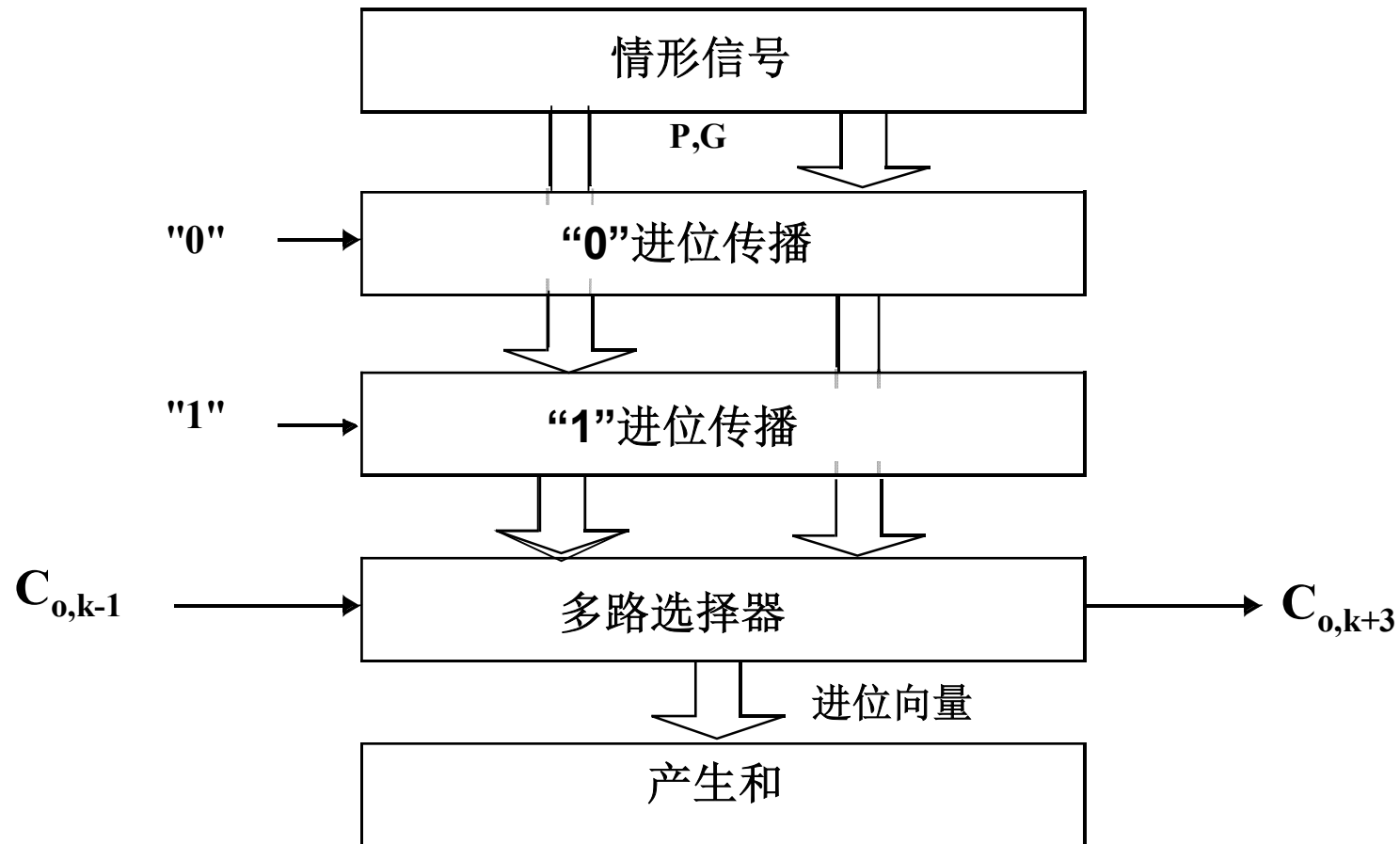
- 增加进位旁路一般使面积增加**10% 至20%**
- 进位旁路加法器的总进位传播时间仍与位数 N 成正比，但比例系数比逐位进位加法器为小。
- N 较小时，旁路的额外开销使采用旁路进位的收益不大。一般旁路进位和行波进位的交叉点位于 N 在**4~8** 之间的范围。



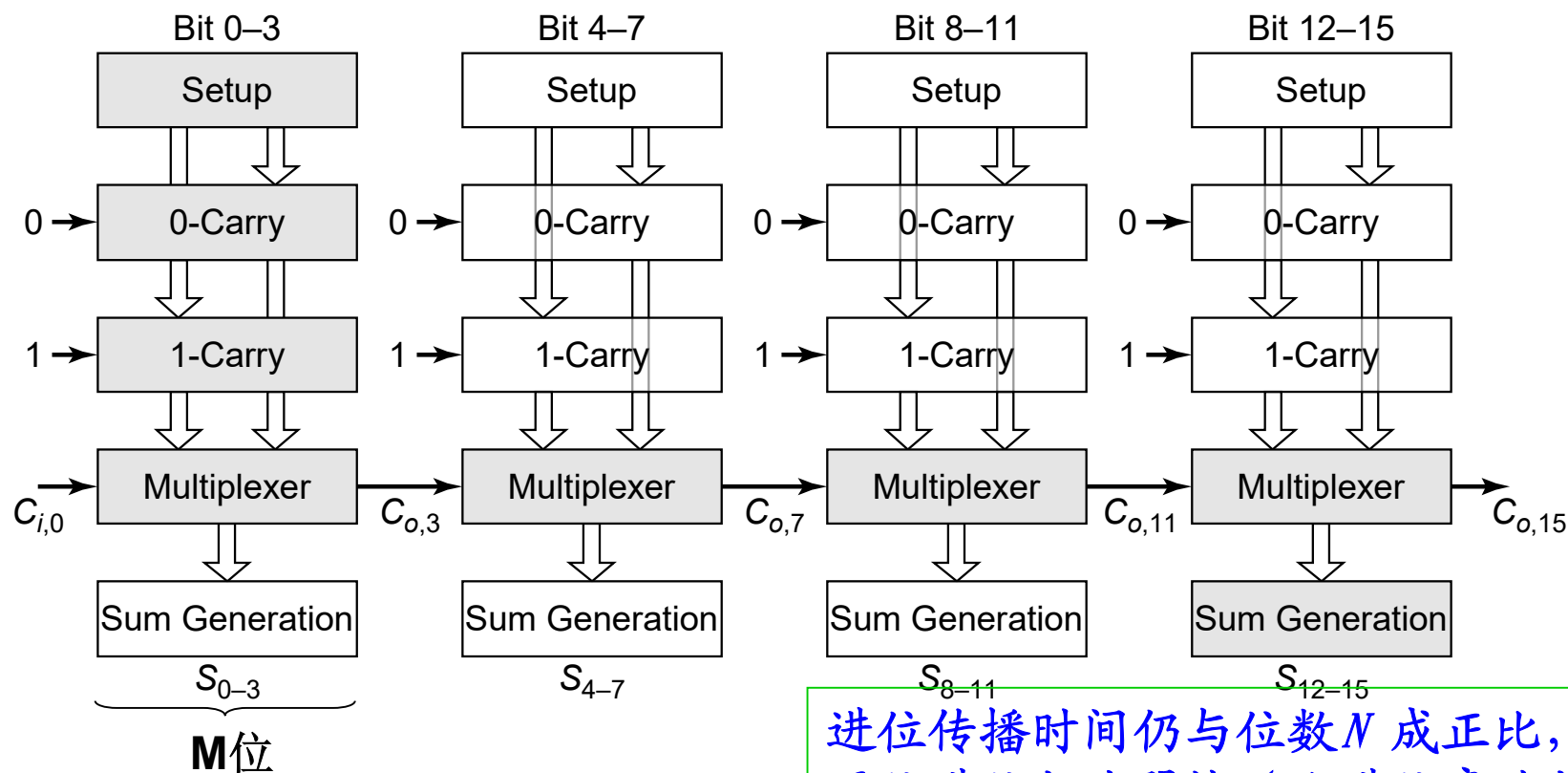
加法器设计

- ❖ 二进制加法器
- ❖ 改进串行进位加法器
- ❖ 进位选择加法器
- ❖ 超前进位加法器
- ❖ 树形加法器

进位选择加法器



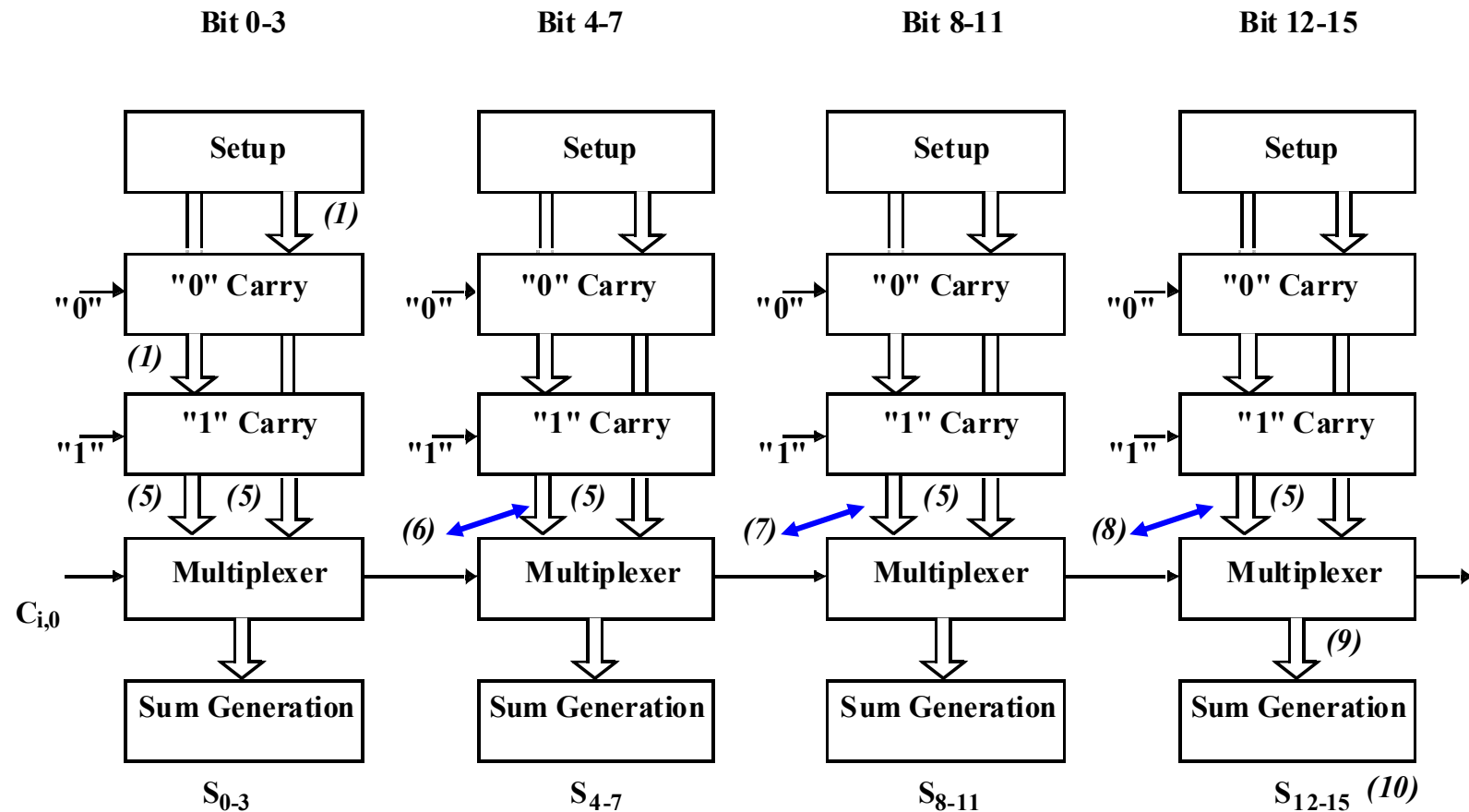
进位选择加法器：关键路径



$$t_{add} = t_{setup} + Mt_{carry} + \left(\frac{N}{M} \right) t_{mux} + t_{sum}$$

进位传播时间仍与位数 N 成正比，但比逐位进位加法器快（比进位旁路加法器少 $M-1$ 级进位传输延时）。
硬件开销为一个额外的进位路径和一个 MUX，大约比一个逐位进位加法器多 30%

线性进位选择加法器

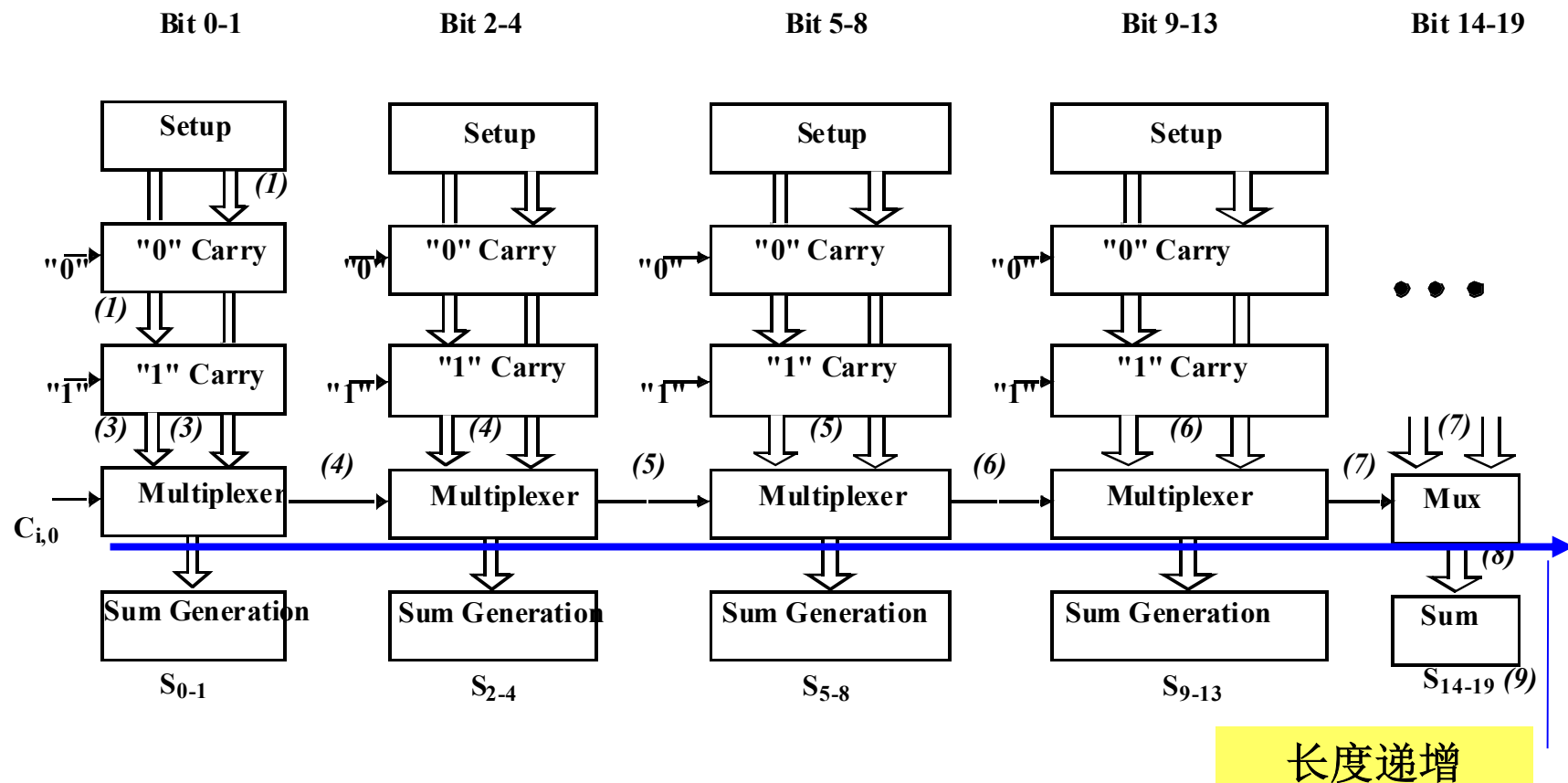


$$t_{add} = t_{setup} + Mt_{carry} + \left(\frac{N}{M}\right)t_{mux} + t_{sum}$$

存在时间浪费

平方根进位选择加法器

考虑到前级的进位输出要经过一个**MUX**才到达本级的进位输入，因此在两条信号路径之间相差一个延时时间，故本级的位数可以比前一级多一级



平方根进位选择加法器的延时：

假设 N 位的加法器含有 P 个级，且第一级是 M 位，后续级逐级增加一位，于是：

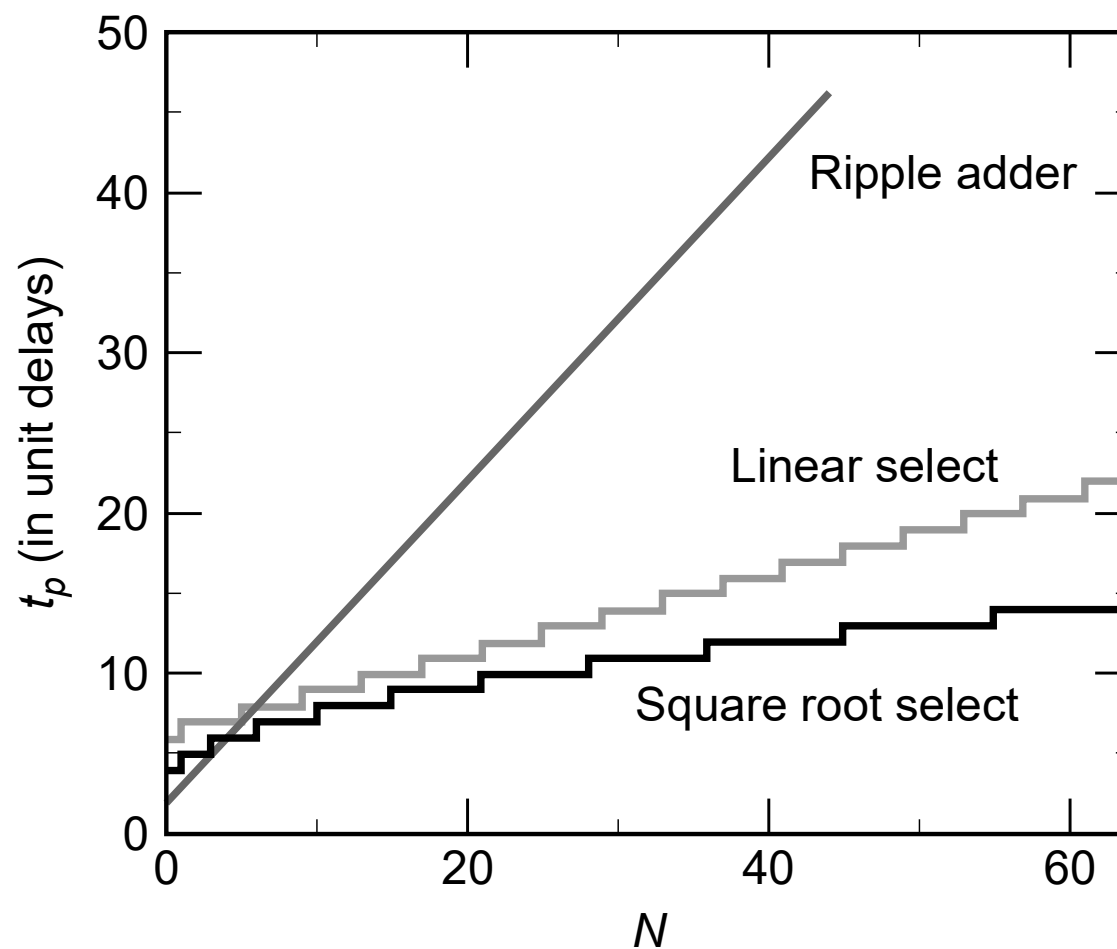
$$\begin{aligned} N &= M + (M+1) + (M+2) + (M+3) + \dots + (M+P-1) \\ &= MP + \frac{P(P-1)}{2} = \frac{P^2}{2} + P(M - \frac{1}{2}) \end{aligned}$$

$$\text{如果 } M \ll N, \text{ 则 } N \approx \frac{P^2}{2} \text{ 或 } P \approx \sqrt{2N}$$

$$t_{add} = t_{setup} + Mt_{carry} + \sqrt{2N}t_{mux} + t_{sum}$$

此时延时正比于 $\sqrt{2N}$ （亚线性关系），而不是 N （线性关系）
当 N 很大时，延时几乎变为常数。

加法器延时比较



加法器设计

- ❖ 二进制加法器
- ❖ 改进串行进位加法器
- ❖ 进位选择加法器
- ❖ 超前进位加法器
- ❖ 树形加法器

超前进位（Carry Lookahead）加法器

🌸 基本原理

$$C_{o,k} = G_k + P_k C_{o,k-1} \quad \text{展开:}$$

$$\begin{aligned} C_{o,k} &= G_k + P_k (G_{k-1} + P_{k-1} (\dots + P_1 (G_0 + P_0 C_{i,0})) \dots) \\ &= \underline{G_k + P_k (G_{k-1} + P_{k-1} (\dots (G_1 + P_1 G_0) \dots))} + \underline{P_k \dots P_1 P_0 C_{i,0}} \\ &\triangleq G_{k:0} + P_{k:0} C_{i,0} \end{aligned}$$

$$S_k = P_k \oplus C_{o,k-1}$$

🌸 由此可知，每一位的进位输出只与最初的进位输入有关，而与比它低位的其他进位输入无关，因此**加法时间与位数 N 无关**。

超前进位加法器的拓扑结构

超前进位加法器(CLA):

直接由情形信号和原始进位输入计算“和”与进位输出。

$$c_{o,3} = G_3 + P_3(G_2 + P_2(G_1 + P_1(G_0 + P_0 C_{in})))$$

其上拉网络可以按照如下方法得到:

$G_i = a_i b_i$ 对应的输入反相形式为

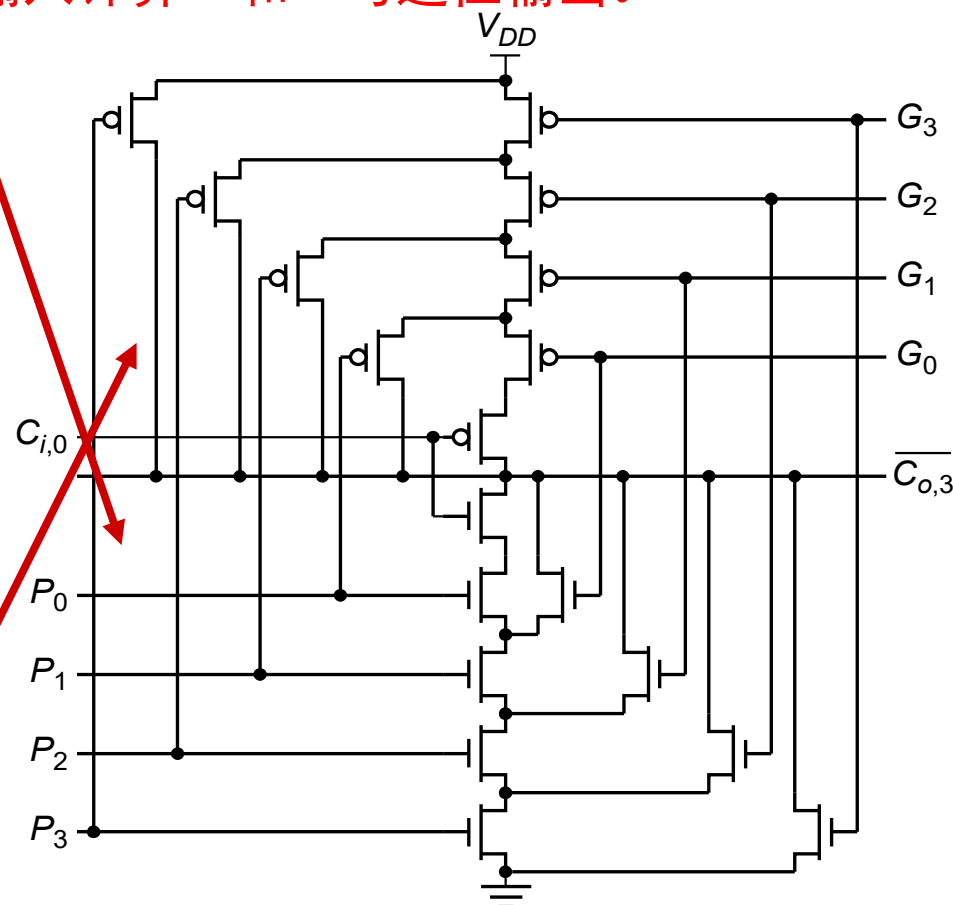
$$\overline{a_i b_i} = \overline{a_i + b_i} = \overline{P_i}$$

$P_i = a_i + b_i$ 对应的输入反相形式为

$$\overline{G_i} = \overline{a_i b_i}$$

因此有:

$$\overline{c_{o,3}} = \overline{P_3 + G_3(P_2 + G_2(P_1 + G_1(P_0 + G_0 C_{in})))}$$



超前进位加法器

超前进位加法器的局限性：

当位数 N 较大时，

- 电路的大扇入会使速度变慢，如采用较简单的门来实现，就会要求较多的逻辑层次。
- 某些信号的扇出会很大 (P_0, G_0)，使加法器的延时加大。
- 实现面积也会随位数 N 的增加而增加。

因此这一超前进位只对小的 N 值（ $N \leq 4$ ）有效。

实际当中，很少使用超过4位的CLA，但是它所蕴含的关系却可以被用来实现高速的加法器。

加法器设计

- ❖ 二进制加法器
- ❖ 改进串行进位加法器
- ❖ 进位选择加法器
- ❖ 条件求和加法器
- ❖ 超前进位加法器
- ❖ 树形加法器

块情形信号

$$C_{o,0} = G_0 + P_0 C_{i,0}$$

$$C_{o,1} = G_1 + P_1 G_0 + P_1 P_0 C_{i,0} = \underline{(G_1 + P_1 G_0)} + \underline{P_1 P_0} C_{i,0} = \underline{G_{1:0}} + \underline{P_{1:0}} C_{i,0}$$

$$C_{o,2} = G_2 + P_2 C_{o,1} = (G_2 + P_2 G_{1:0}) + P_2 P_{1:0} C_{i,0} = G_{2:0} + P_{2:0} C_{i,0}$$

$$C_{o,3} = G_3 + P_3 C_{o,2} = G_3 + P_3 (G_2 + P_2 C_{o,1}) = \underline{G_3} + \underline{P_3 G_2} + \underline{P_3 P_2} C_{o,1} = \underline{G_{3:2}} + \underline{P_{3:2}} C_{o,1}$$

...

$$\begin{aligned} C_{o,i} &= G_i + P_i C_{o,i-1} = G_i + P_i (G_{i-1:j} + P_{i-1:j} C_{o,j}) = (G_i + P_i G_{i-1:j}) + P_i P_{i-1:j} C_{o,j} \\ &= \underline{G_{i:j}} + \underline{P_{i:j}} C_{o,j} \end{aligned}$$

$$G_{i:j} = G_i + P_i (G_{i-1} + \dots (G_{j+2} + P_{j+2} (G_{j+1} + P_{j+1} G_j)) \dots)$$

$$P_{i:j} = \prod_{k=j}^i P_k$$

$$G_{i:j} = G_{i:k} + P_{i:k} G_{k-1:j}, P_{i:j} = P_{i:k} \cdot P_{k-1:j}, i < k < j$$

$$C_i = G_{i:j} + P_{i:j} C_j$$

块情形信号是位情形信号的一般化。

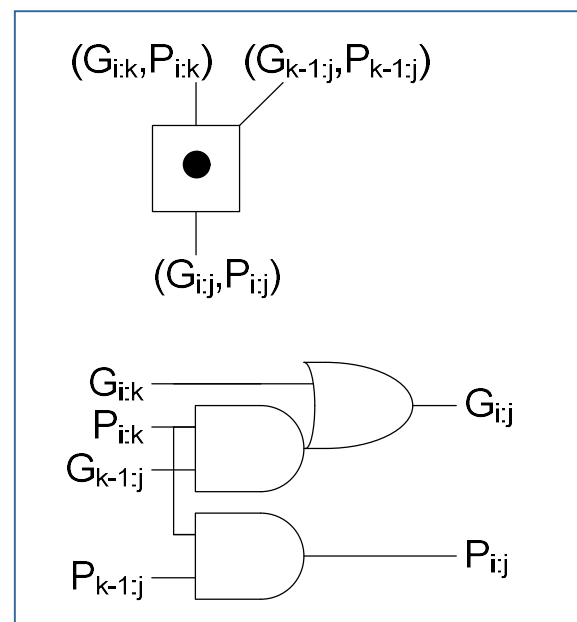
前缀算子

定义： 前缀 (**prefix**) 算子, 又称点算子、多数 (**majority**) 算子、进位算子等

$$(G, P) \bullet (G', P') = (G + PG', PP')$$

前缀算子满足结合律

$$(G_{i:j}, P_{i:j}) = (G_{i:k}, P_{i:k}) \bullet (G_{k-1:j}, P_{k-1:j}), i > k > j$$



$$\begin{aligned} (G_{i:j}, P_{i:j}) &= (G_i, P_i) \bullet (G_{i-1:j}, P_{i-1:j}) \\ &= (G_i, P_i) \bullet (G_{i-1}, P_{i-1}) \dots \bullet (G_j, P_j) \\ &= ((G_i, P_i) \bullet (G_{i-1}, P_{i-1})) \dots ((G_k, P_k) \dots \bullet (G_j, P_j)) \end{aligned}$$

进位计算问题

重新定义 $G_{0:0} = C_{in}$, 则

$$G_{i:i} \equiv G_i = A_i B_i, P_{i:i} \equiv P_i = A_i \oplus B_i$$

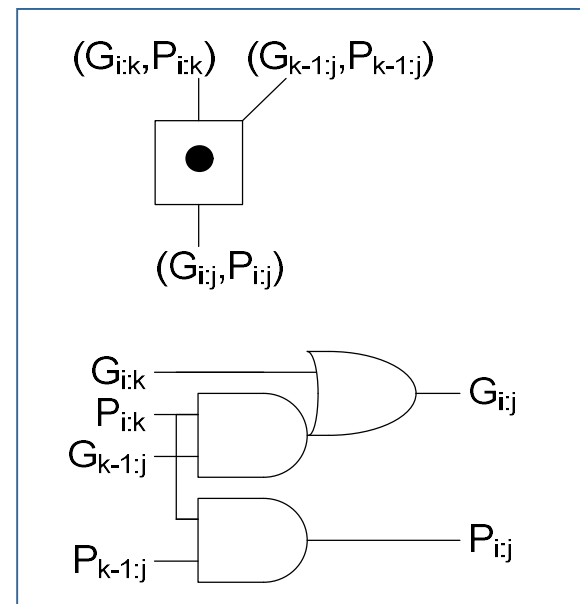
$$S_i = P_i \oplus G_{i-1:0}$$

$$C_{out} = G_{N:0} = G_N + P_N G_{N-1:0}$$

加数A, B的各位从1开始编号。

进位计算问题:

$$\begin{aligned}(G_{i:0}, P_{i:0}) &= (G_i, P_i) \bullet (G_{i-1:0}, P_{i-1:0}) \\ &= (G_i, P_i) \bullet (G_{i-1}, P_{i-1}) \dots \bullet (G_0, P_0)\end{aligned}$$



N位加法器的问题抽象

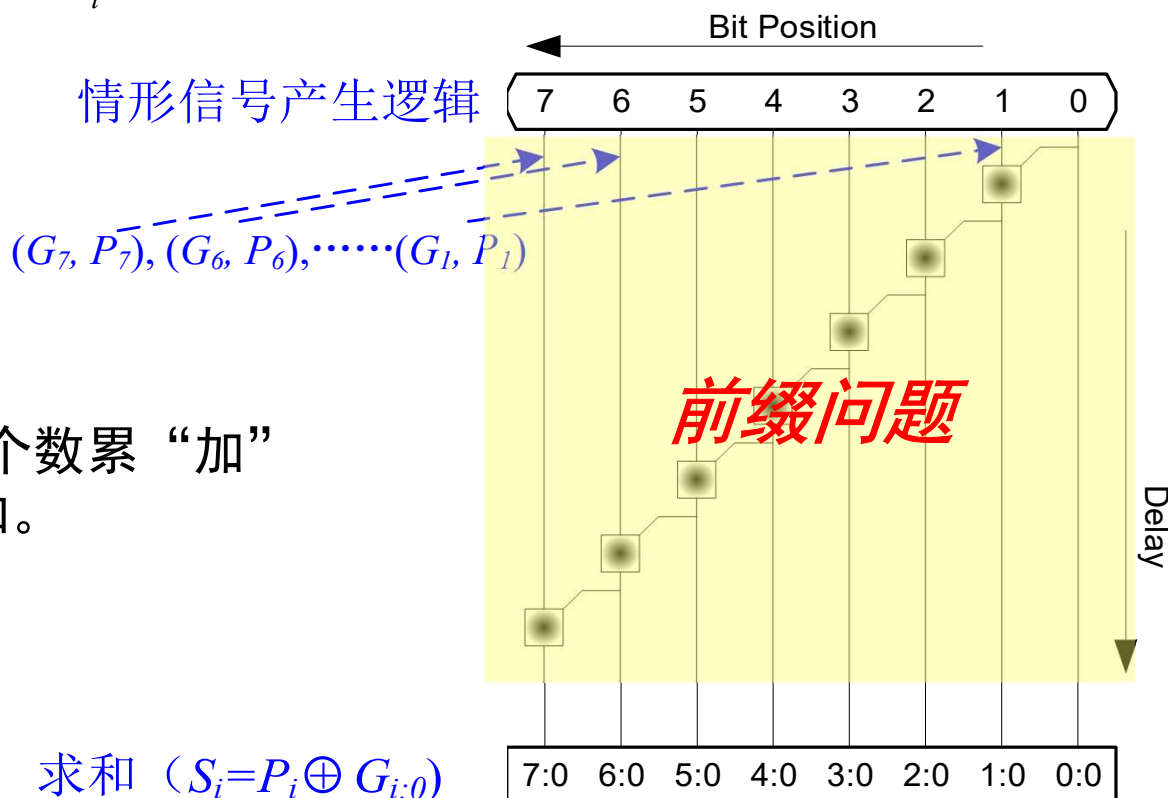
$$G_{0:0} = C_{in}$$

$$G_{i:i} \equiv G_i = A_i B_i, P_{i:i} \equiv P_i = A_i \oplus B_i$$

$$S_i = P_i \oplus G_{i-1:0}$$

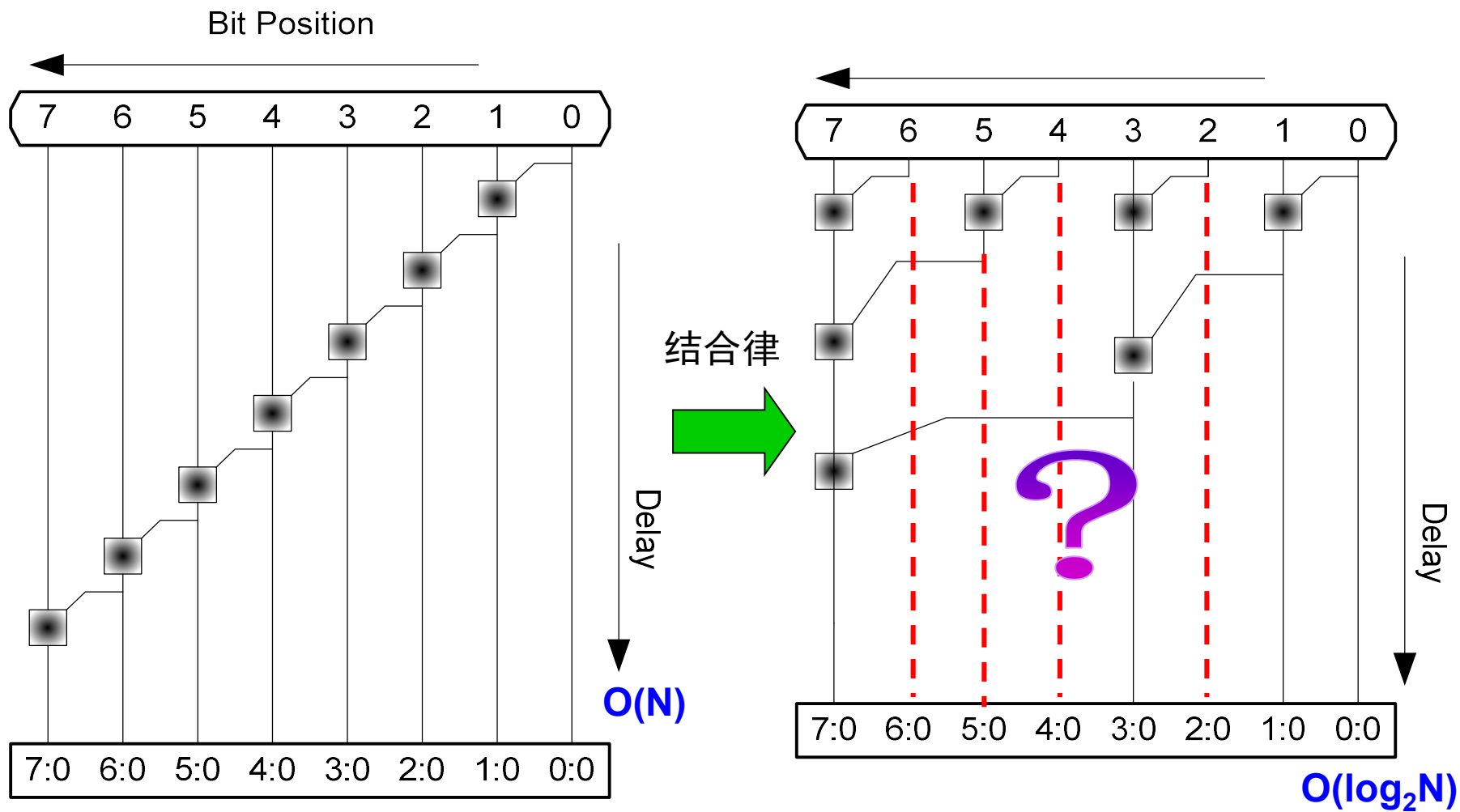
$$C_{out} = G_{N:0} = G_N + P_N G_{N-1:0}$$

前缀(prefix)问题：计算n个数累“加”过程的所有中间值和最终和。



对数prefix结构

a8

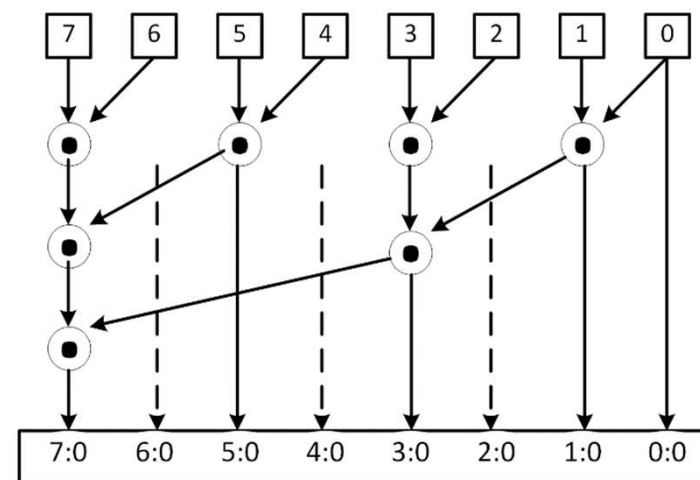
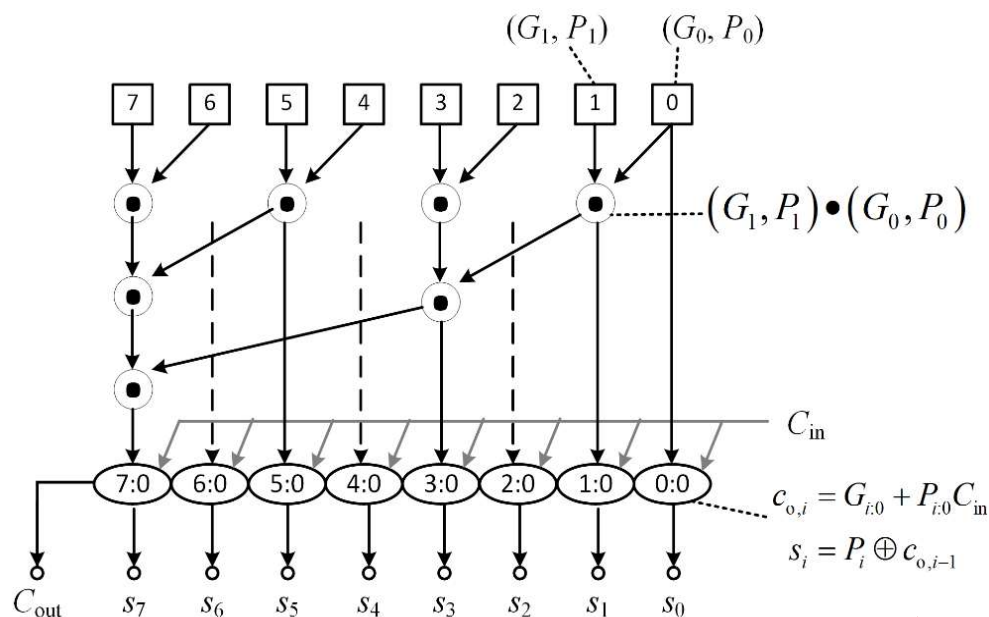


a8

对于 $G7:0$ 可以根据结合律通过2分分组的方法在对数时间内完成，在此过程中还可以得到 $G1:0$ 和 $G3:0$ ，以此类推， $G2^{n-1}:0$ 的点运算都可以按此方法得到，但是prefix问题还要求求解非 2^{n-1} 位置上的输出（图中红色虚线标出的位）。

adm, 2011/12/11

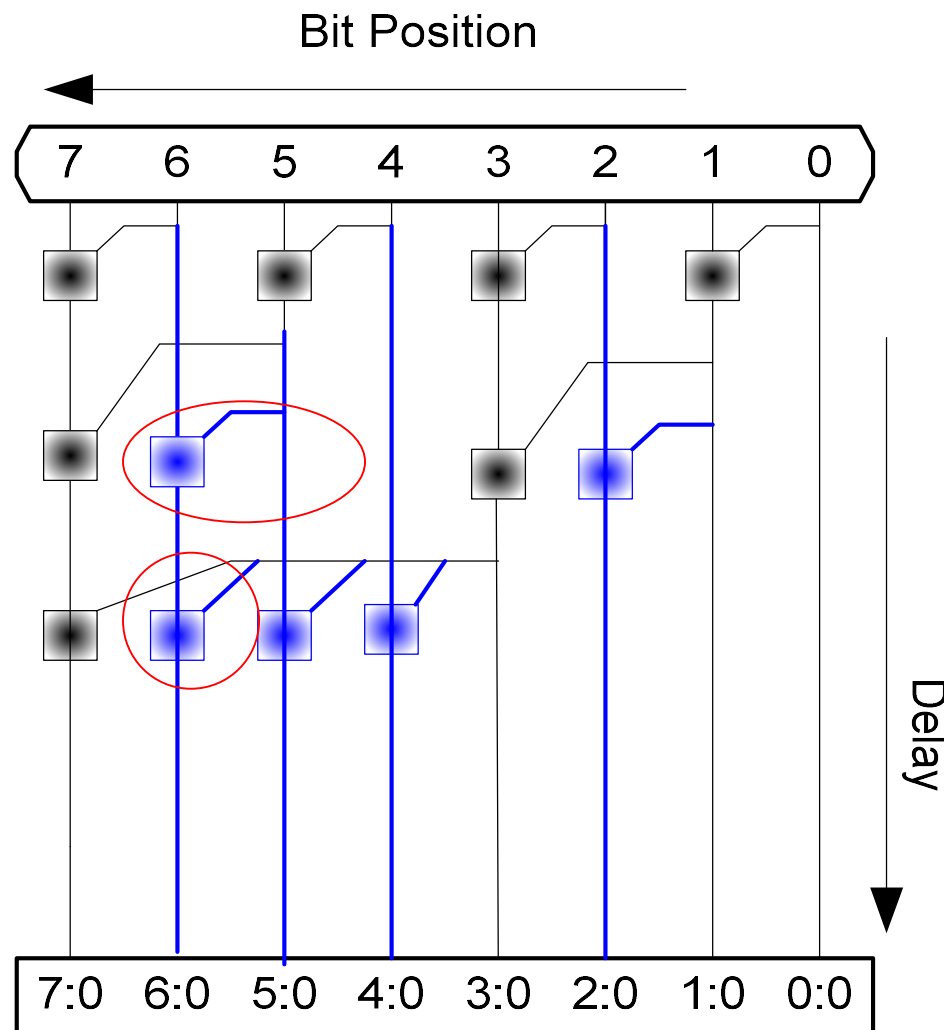
另一种情形信号序号方法



$$t_{\text{adder-tree}} = t_{\text{setup}} + t_{\text{prefix}} + t_{\text{ca-sum}}$$

Sklansky树结构

LX3



中间位的块情形
信号和2的幂次
位的块情形信号
一起产生。

$$D(N)=\log_2 N$$

LX3

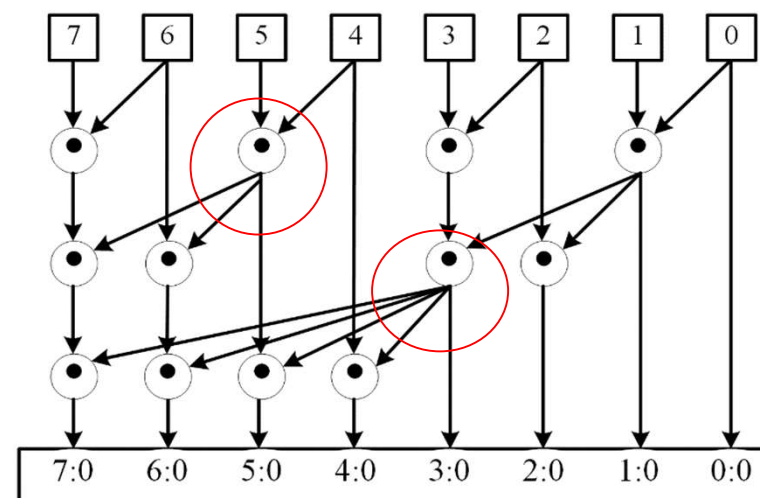
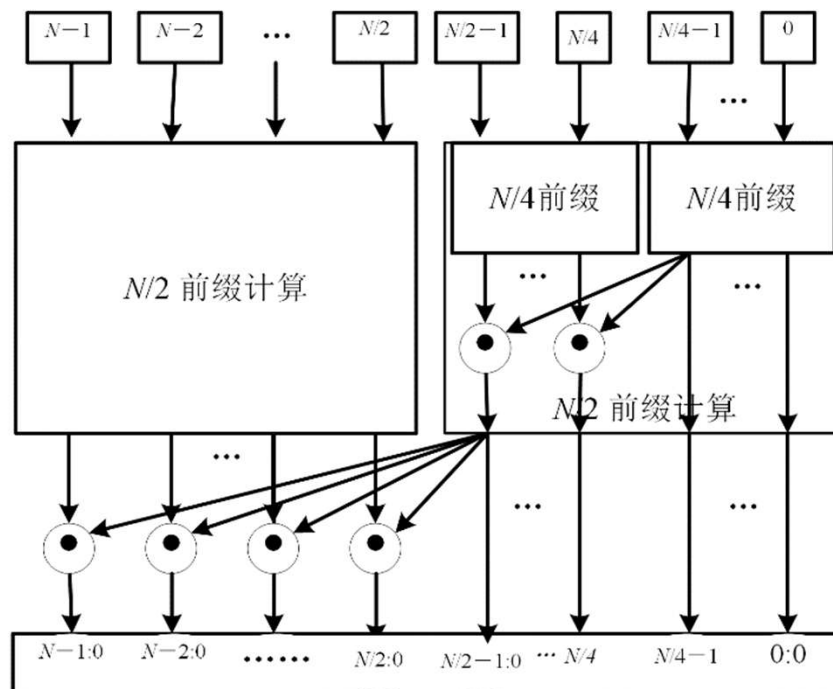
又称分治树，将 $N:0$ 的前置问题分解成 $N:N/2$ 的前置问题和 $N/2-1:0$ 的前置问题，把 $N:N/2$ 的前置问题的结果与 $(G_{(N/2-1)}, P_{(N/2-1)})$ 组会产生对应的进位输出；再把 $N:N/2$ 的前置问题和 $N/2-1:0$ 的前置问题都分解成两个 $N/4$ 的前置问题，逐层递归到 $N=1$ ，即得到Sklansky树。

Li XY, 2015/12/15

Sklansky树结构——原理说明

LX4

高低划分的分治树形前缀计算



$$D(N) = \log_2 N$$

中间位的块情形信号和2的幂次位的块情形信号一起产生。

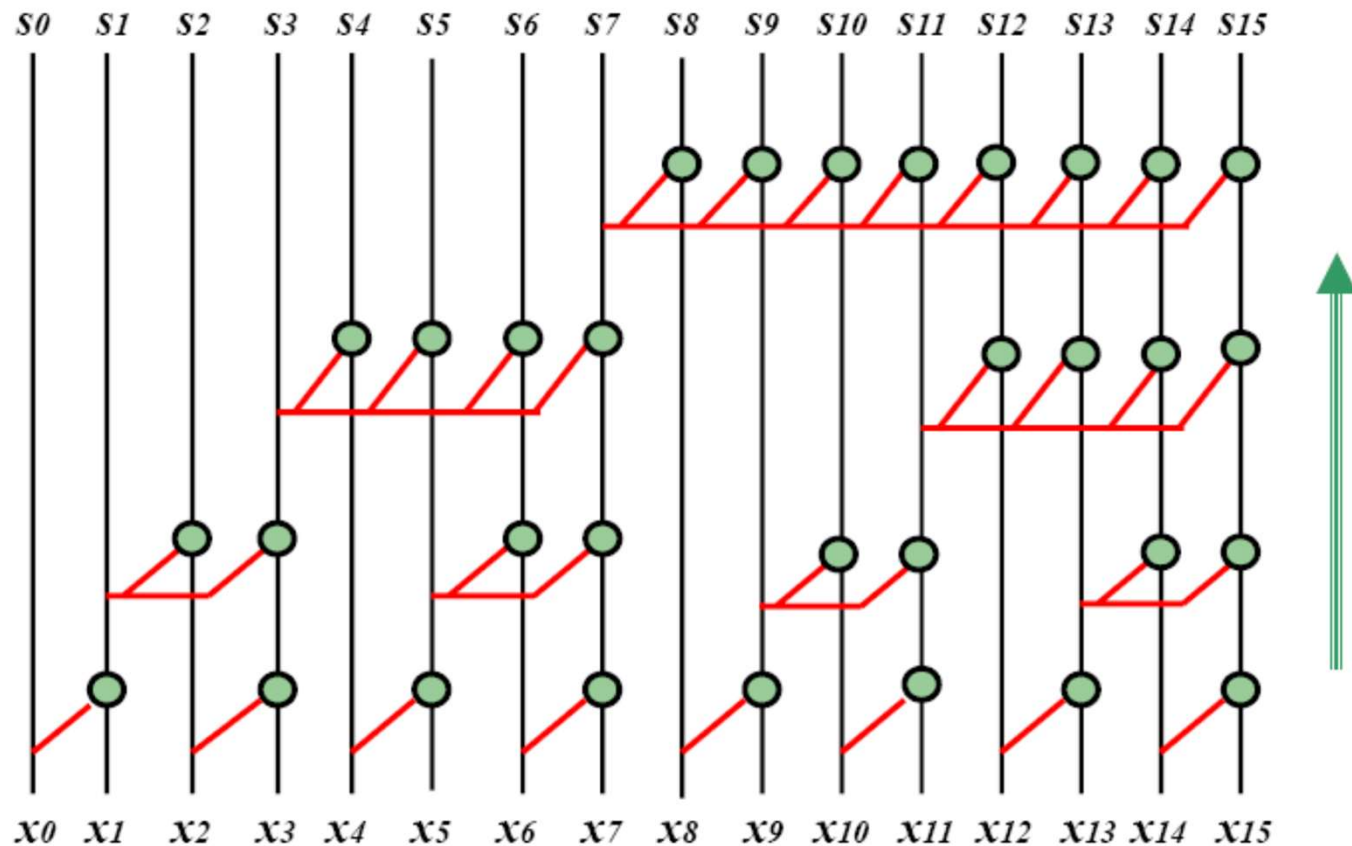
LX4

又称分治树，将 $N:0$ 的前置问题分解成 $N:N/2$ 的前置问题和 $N/2-1:0$ 的前置问题，把 $N:N/2$ 的前置问题的结果与 $(G_{(N/2-1)}, P_{(N/2-1)})$ 组会产生对应的进位输出；再把 $N:N/2$ 的前置问题和 $N/2-1:0$ 的前置问题都分解成两个 $N/4$ 的前置问题，逐层递归到 $N=1$ ，即得到Sklansky树。

Li XY, 2015/12/15

Sklansky 加法器

把**16**位加法分解成两个**8**位的进位计算，高**8**位的进位由自身的子**prefix**结果和**7: 0**位的块情形信号产生，.....依次递归。

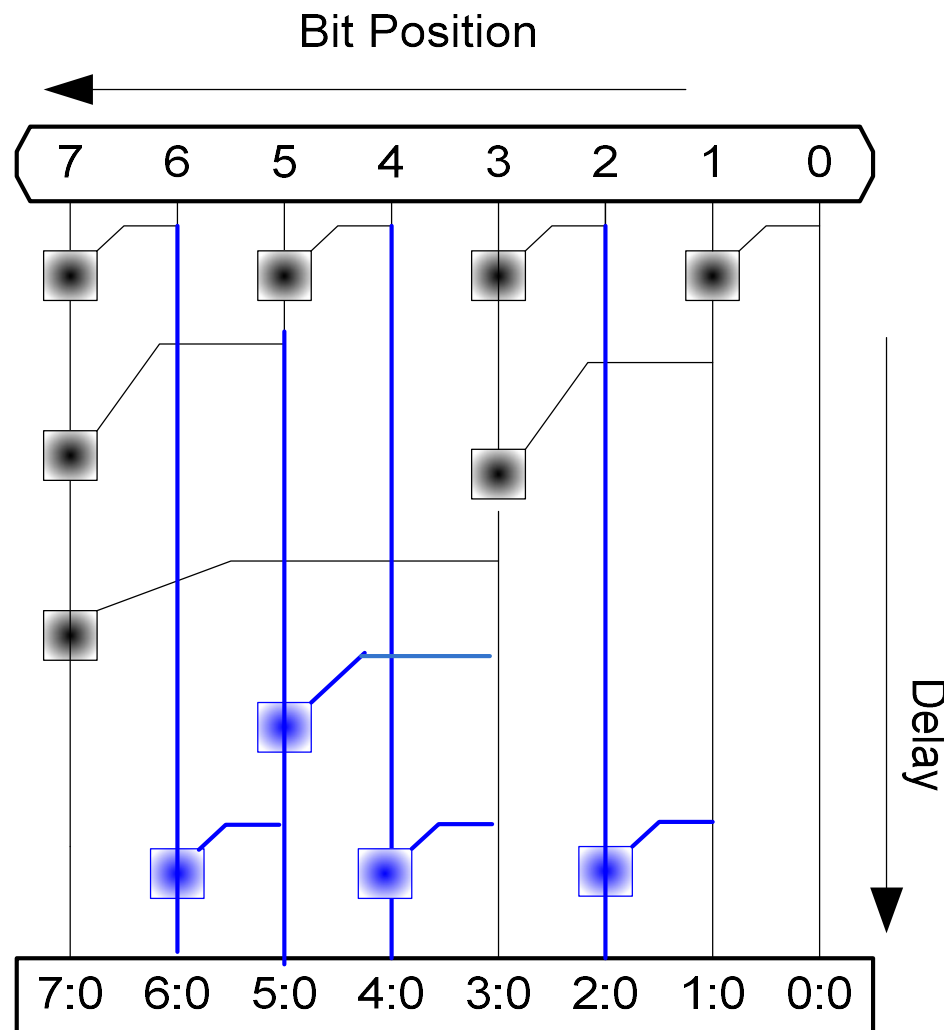


Sklansky 加法器的特点

- ❖ 延时与位数成对数关系
- ❖ 每级前缀算子的扇出增加，会带来性能下降。

Brent-Kung树结构

LX6



用反向二进制树
(fan back tree)产
生其余位的进位
信号。

$$D(N)=2\log_2 N-2$$

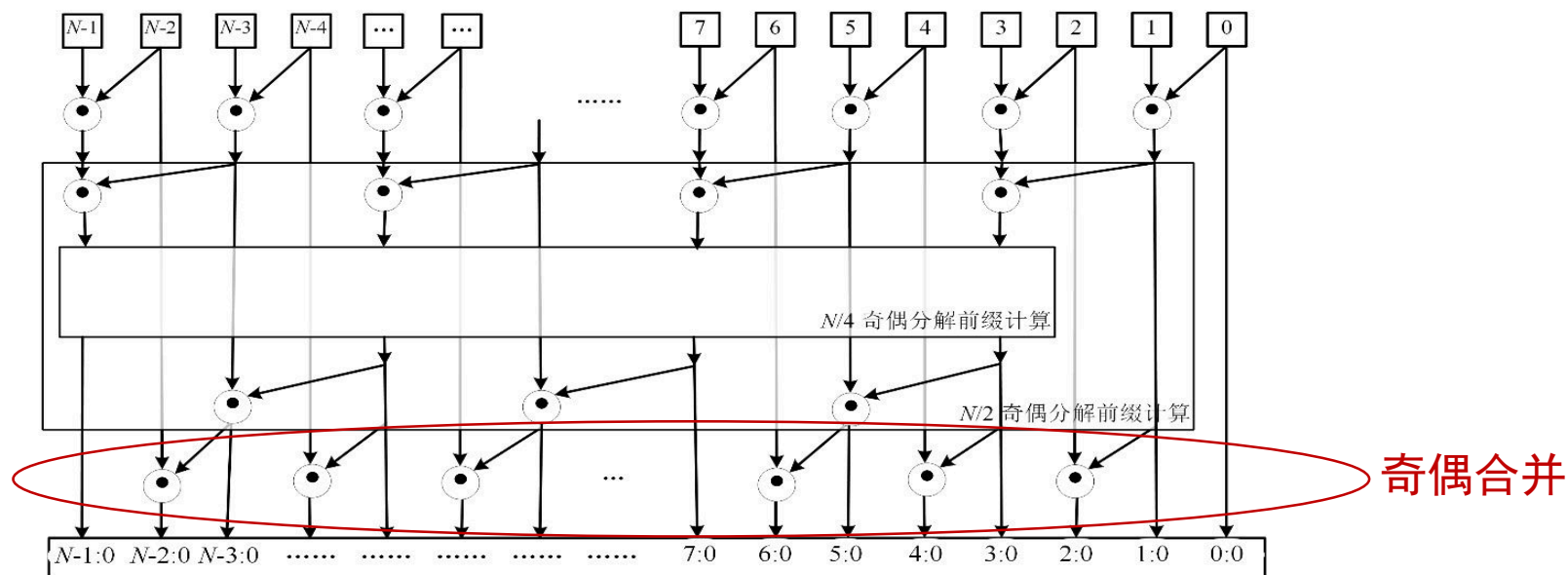
LX6

计算剩余位进位输出有多种结构，brent-kung结构就是其中一种。

对于剩余位，按照每层都两两组合的方式，由底层向上层合并，形成反向的二进制数。

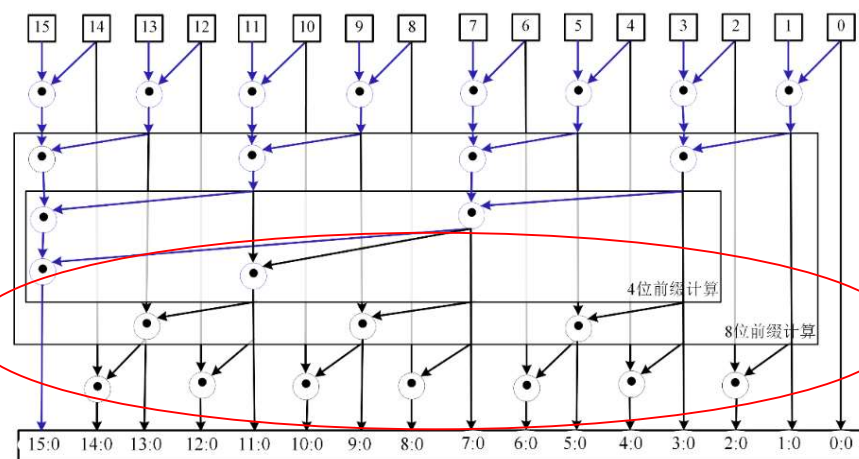
Li XY, 2015/12/15

Brent-Kung树结构——原理说明

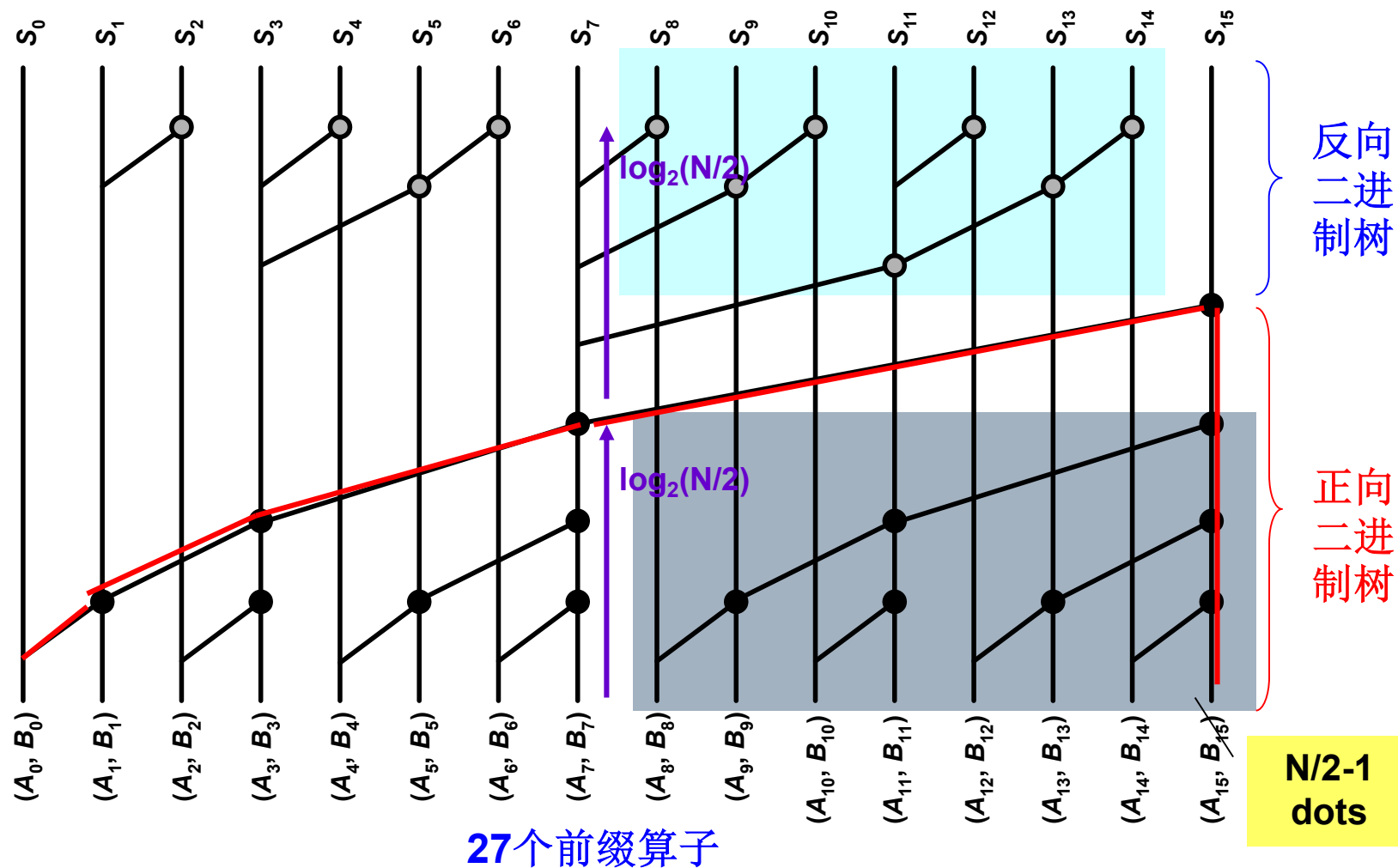


奇偶分治树形加法器

用反向二进制树(fan back tree)
产生其余位的进位信号。



Brent-Kung Tree

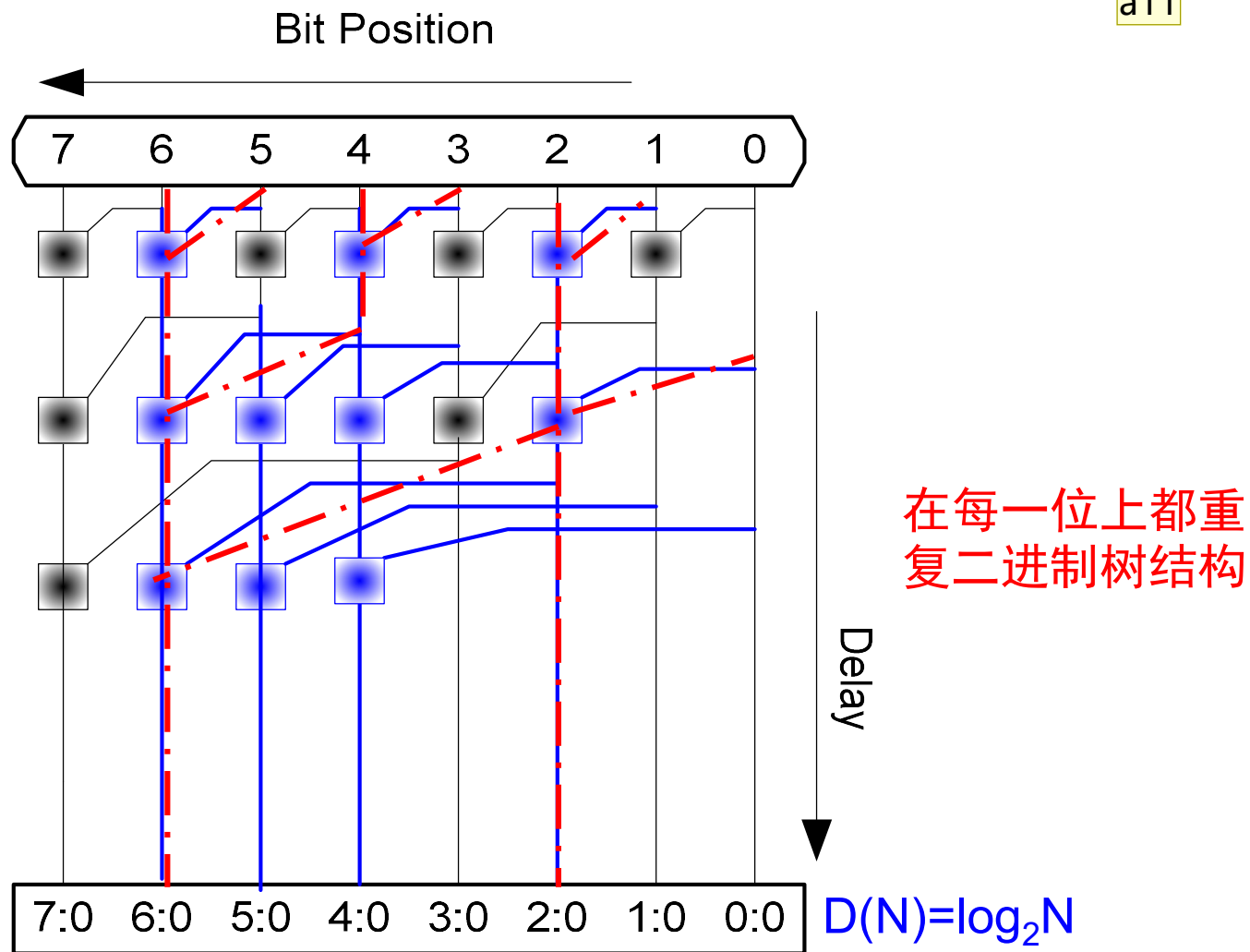


Brent-Kung树的特点

- ❖ 结构简单、门数少（面积小、功耗低）
- ❖ 关键路径长，最多（ S_{14} ）经过6级前缀算子
- ❖ 结构不规则，门的尺寸确定困难

Kogge-Stone树结构

a11



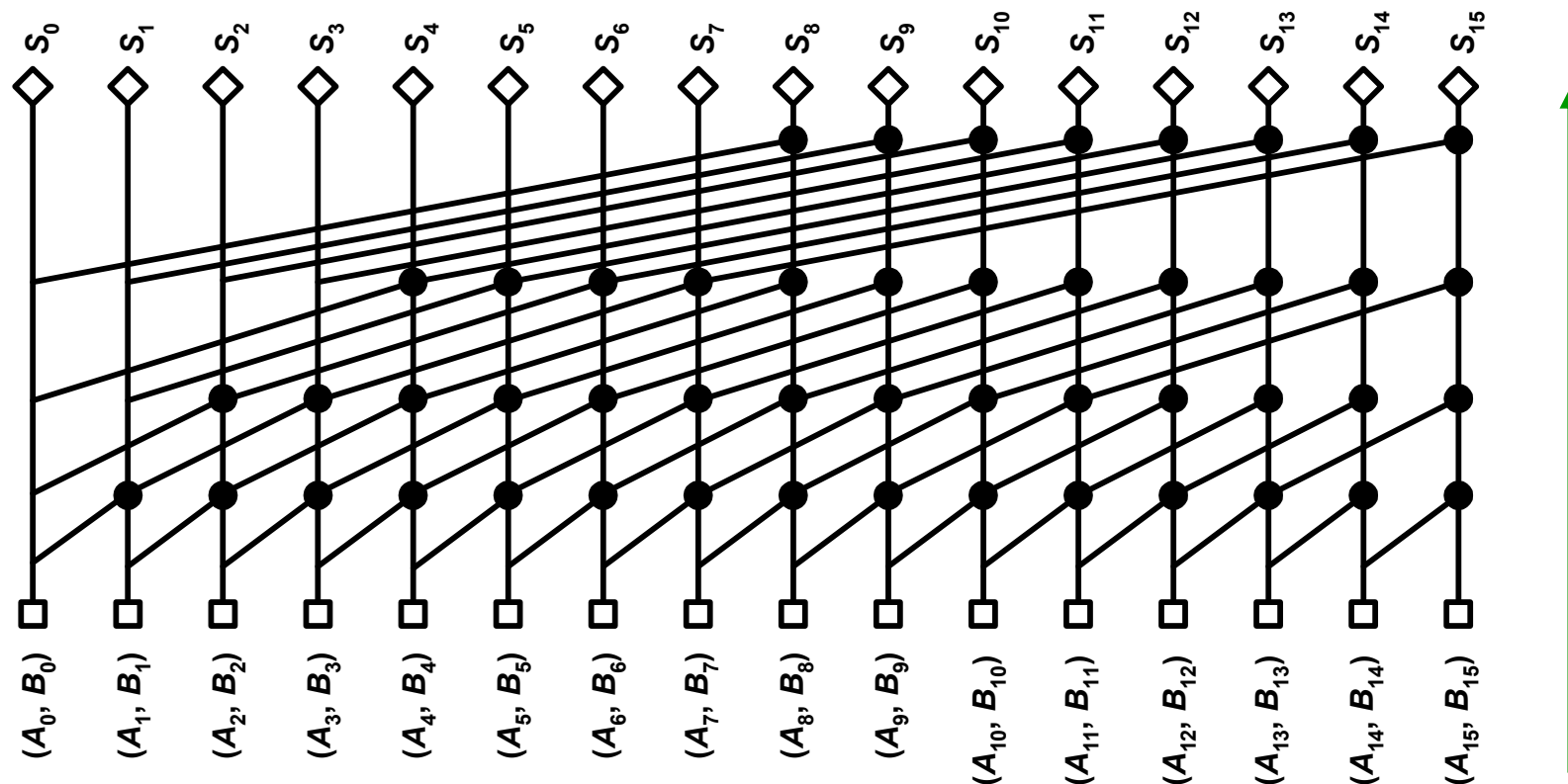
幻灯片 41

a11

红色点间线标出的树形结构与2的幂次位组成的结构相同

adm, 2011/12/11

Kogge-Stone树形加法器



16-bit radix-2 Kogge-Stone tree

49个逻辑门

Kogge-stone加法器的特点

- ❖ 逻辑层数正比于 $\log_2 N$ 因此传播延时也正比于 $\log_2 N$
- ❖ 结构规则，关键路径扇出较小而且基本是一个常数，确定尺寸容易
- ❖ 版图紧凑（一定程度上可以消除门数较多带来的面积开销）
- ❖ 门数较多
- ❖ 各级之间有很多长连线

广泛应用于高性能的32位加法器和64位加法器。

树形结构小结

- ❖ 对于二进制树，延时 $t_p = t_{setup} + \lceil \log_2 N \rceil t_{prefix} + t_{xor}$
- ❖ Sklansky 的扇出太大
- ❖ Kogge-stone的连线太多
- ❖ Brent-Kung的逻辑级数太多
- ❖ 存在一些上述三种树形结构相结合的树形结构，它们对扇出、连线复杂度和逻辑深度进行折中。

加法器小结

🔥 结构:

- **Ripple (逐位进位) 加法器:**
 1. 面积小, 简单, 速度相对可以接受, 应首先考虑使用。
 2. 易改成流水线加法器。
- **By-pass (进位旁路)加法器:**
 - 速度较快, 面积较大
- **Carry Look Ahead (CLA) 加法器:**
 1. 非常快的非流水线加法器。(连线不规则)
 2. 功耗相对较大。
 3. $N < 4$
- **Carry-Select Adder:**
 1. 较快的 n 位加法器。
 2. 容易用Ripple Adder 和MUX 组装。
 3. 面积较大。

加法器小结(续)

结构（续）：

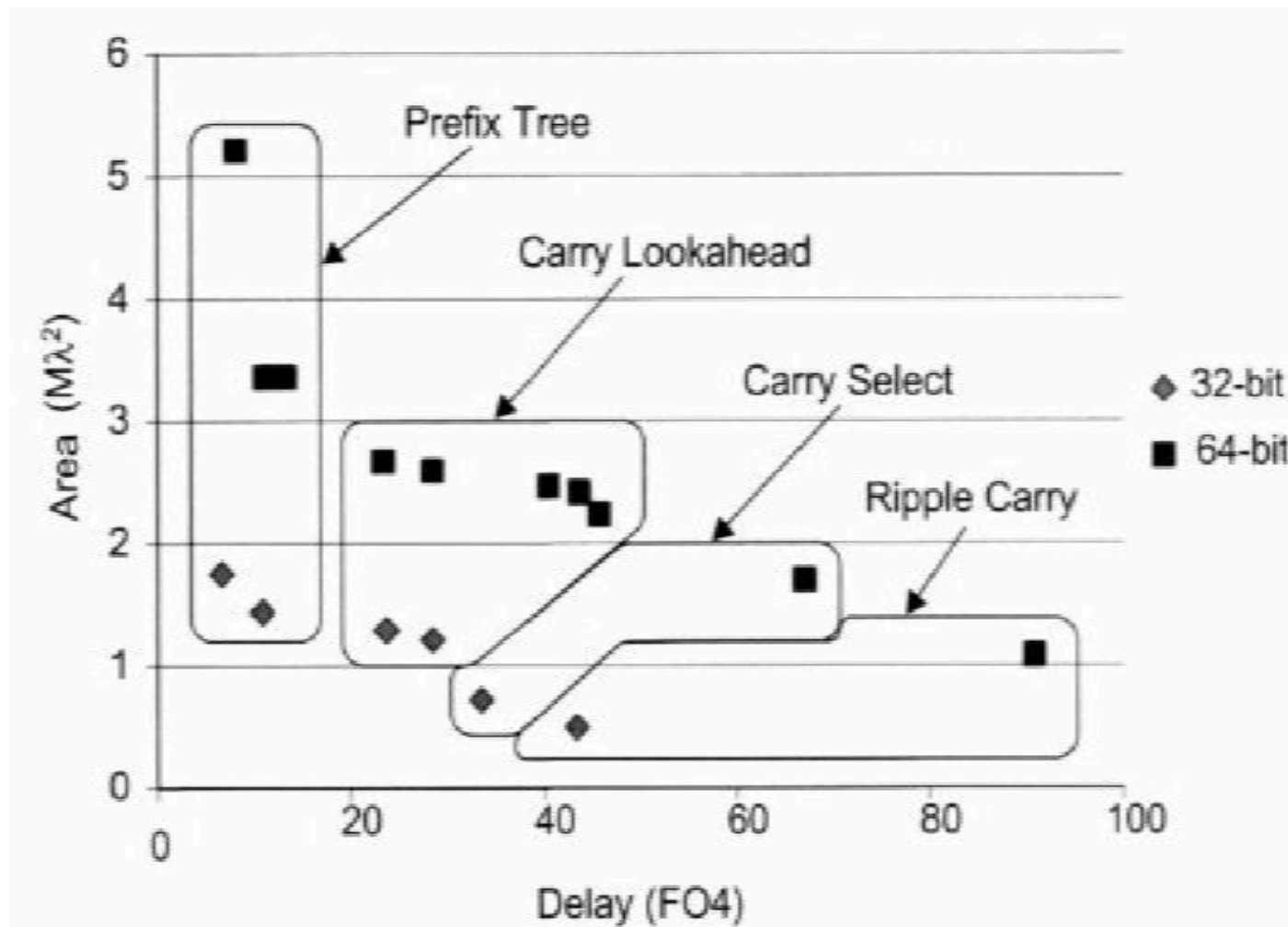
- 树形加法器（prefix）

- 速度快（对数加法器）、规模大，适用于大的加法器
- 在连线复杂度、速度、扇出之间存在着不同的折中
- Brent-Kung加法器的版图面积可以很小，Kogge-Stone加法器由于结构规则、扇出小，可以获得很高的性能。

电路实现：

- 传输门加法器：用在要求进位与和的传播时间必须是接近的情形。（如在乘法阵列中）。
- Manchester 加法器：
 - 1. 规则，较小，较快。
 - 2. 适合于全定制设计的数据通路，字宽从16~ 32位。

32位/64位加法器的延时和面积（综合结果）



总结

加法器结构提出了面积/功耗/延时间的折中问题，根据应用选择最合适的结构。

结构	逻辑级数	最大扇出	布线通道	单元数
Carry-Ripple	$N-1$	1	1	N
Carry-Skip (4段)	$N/4 + 5$	2	1	$1.25N$
平方根进位选择.	$\sqrt{2N}$	$\sqrt{2N}$	1	$2N$
Brent-Kung	$2\log_2 N - 2$	2	1	$2N$
Sklansky	$\log_2 N$	$N/2 + 1$	1	$0.5 N \log_2 N$
Kogge-Stone	$\log_2 N$	2	$N/2$	$N \log_2 N$



- 加法器设计

- 乘法器设计

第十章

数据通路设计

乘法器

二进制乘法器

-  部分积产生
-  部分积的累加
-  最终相加
-  高性能乘法器结构

二进制乘法

M位×N位

$$X = \sum_{i=0}^{M-1} X_i 2^i$$

$$Y = \sum_{j=0}^{N-1} Y_j 2^j$$

$$\begin{aligned} Z &= X \times Y = \sum_{k=0}^{M+N-1} Z_k 2^k \\ &= (\sum_{i=0}^{M-1} X_i 2^i) (\sum_{j=0}^{N-1} Y_j 2^j) \\ &= \sum_{i=0}^{M-1} (\sum_{j=0}^{N-1} \underline{X_i Y_j} 2^{i+j}) \end{aligned}$$

AND

	1	0	1	0	1	0	被乘数(Multiplicand)
x			1	0	1	1	乘数 (Multiplier)
					1	0	} 部分积 (Partial products)
		1	0	1	0	1	
	1	0	1	0	1	0	
	0	0	0	0	0	0	
+	1	0	1	0	1	0	} 结果
	1	1	1	0	0	1	
	1	1	1	0	0	1	1

LI X. Y.5

$$XY = x_{m-1}y_{n-1}2^{m+n-2} + \sum_{i=0}^{i=n-2} \sum_{j=0}^{j=m-2} x_j y_i 2^{m+n-4} - (x_{m-1} \sum_{i=0}^{n-2} y_i 2^{m+i-1} + y_{n-1} \sum_{i=0}^{m-2} x_i 2^{n+i-1})$$

2025-2026秋

LI X. Y.5

为了保证编码后的结果仍在 $m-1$ 位2进制编码表示范围内，对于负数要加上 2^{m-1} 后再编码，此时得到的编码结果是相反数的补码，其值等于直接转换到十进制后的结果减掉 2^{m-1} 。

Li Xiangyu, 2020/12/13

有符号数的二进制乘法

$$X = -x_{m-1}2^{m-1} + \sum_{i=0}^{m-2} x_i 2^i, Y = -y_{n-1}2^{n-1} + \sum_{i=0}^{n-2} y_i 2^i \quad \text{符号位 (MSB) 有负的权重}$$

$$XY = x_{m-1}y_{n-1}2^{m+n-2} + \sum_{i=0}^{i=n-2} \sum_{j=0}^{j=m-2} x_j y_i 2^{m+n-4} - (x_{m-1} \sum_{i=0}^{n-2} y_i 2^{m+i-1} + y_{n-1} \sum_{i=0}^{m-2} x_i 2^{n+i-1})$$

				x₃	x₂	x₁	x₀	
				y₃	y₂	y₁	y₀	
					x₂ y₀	x₁ y₀	x₀ y₀	
					x₂ y₁	x₁ y₁	x₀ y₁	
					x₂ y₂	x₁ y₂	x₀ y₂	
				x₃ y₃	1			<u>1</u>
				<u>1</u> <u>1</u>	<u>x₂ y₃</u>	<u>x₁ y₃</u>	<u>x₀ y₃</u>	<u>1</u> <u>1</u> <u>1</u>
				<u>1</u> <u>1</u>	<u>x₃ y₂</u>	<u>x₃ y₁</u>	<u>x₃ y₀</u>	<u>1</u> <u>1</u> <u>1</u>
+	1							<u>1</u>
	S₆	S₅	S₄	S₃	S₂	S₁	S₀	

把部分积中的常数位相加后可以得到如图所示的结果。

Li Xiangyu, 2020/12/13

预计算后的有符号数乘法

modified Baugh-Wooley 乘法器

$$\begin{array}{r}
 \begin{array}{ccccccc}
 & & & & x_3 & x_2 & x_1 & x_0 \\
 & & & & y_3 & y_2 & y_1 & y_0 \\
 + & & & & & & & \\
 \hline
 & & & 1 & \overline{x_0 y_3} & x_2 y_0 & x_1 y_0 & x_0 y_0 \\
 & & & \overline{x_1 y_3} & x_2 y_1 & x_1 y_1 & x_0 y_1 & \\
 & & & \overline{x_2 y_3} & x_2 y_2 & x_1 y_2 & x_0 y_2 & \\
 + & 1 & x_3 y_3 & \overline{x_3 y_2} & \overline{x_3 y_1} & \overline{x_3 y_0} & & \\
 \hline
 & & s_6 & s_5 & s_4 & s_3 & s_2 & s_1 & s_0
 \end{array}
 \end{array}$$

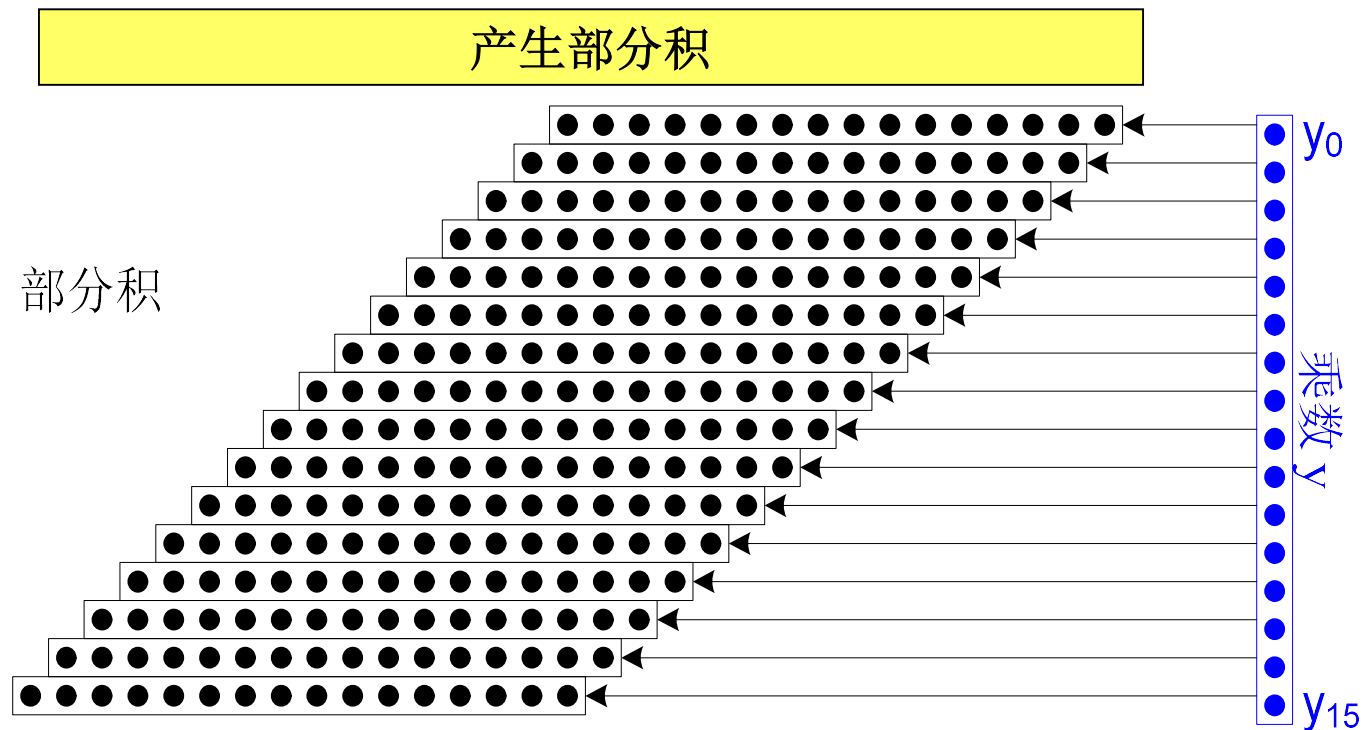
与无符号数乘法器有相似的部分积阵列，设计时可以通过把无符号数乘法器中某些部分积的位用NAND计算，把部分积累加时的闲置输入端置为1实现。因此，有符号数乘法和无符号数乘法可以用相同的阵列结构来实现

把上一页的部分积项重新排列可得到如图所示的排列阵列。注意：相同权重（同一列）的项的位置可以交换

Li Xiangyu, 2020/12/13

乘法器结构

每个点代表部分积的一个比特



阵列乘法器实现:

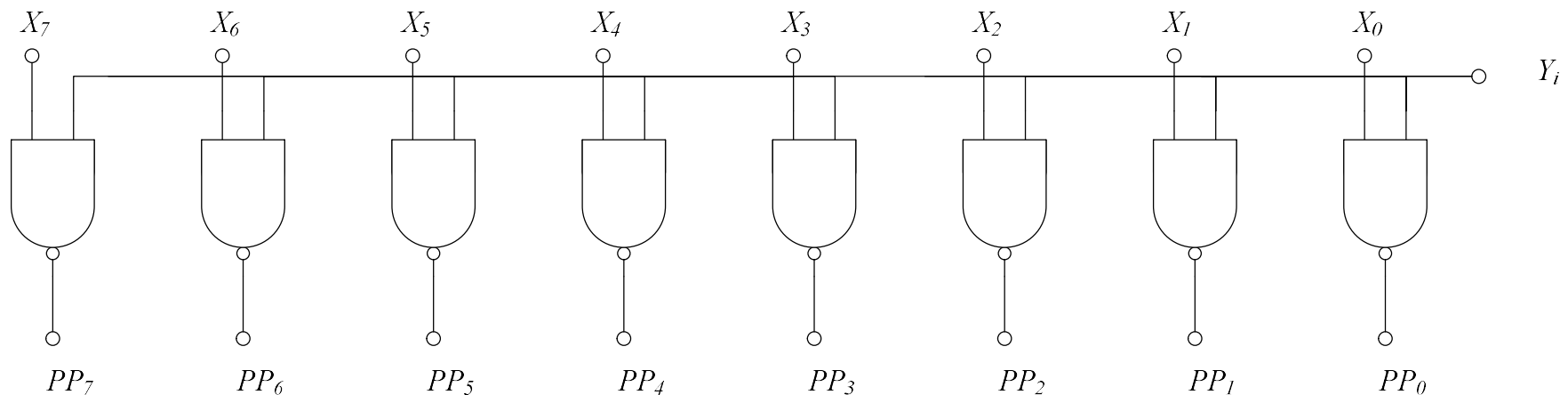
部分积产生-部分积累加-最终相加

乘法器

- ❖ 二进制乘法器
- ❖ 部分积产生
- ❖ 部分积的累加
- ❖ 最终相加
- ❖ 高性能乘法器结构

部分积产生

- 部分积是被乘数 X 和一个乘数位 Y_i 相乘的结果，一位二进制乘法由AND实现。
- 部分积产生逻辑
 - 共 N 个部分积， $M \times N$ 个AND（NAND）



部分积压缩——波兹编码

- ❖ 乘数中等于0的位所对应的部分积对结果没有影响，因此可以只计算非零位的部分积，减少部分积的数目
- ❖ 通过编码的方法可以减少非零行的数目
 - 改进的波兹编码（Booth）：每次乘数中取 k 位（例如 $k=2$ ）与被乘数相乘产生部分积，并在乘数编码中引入-1来减少非零位，称为基 2^k 波兹编码。

$$\underbrace{2^{i+k-1} + 2^{i+k-2} + \cdots + 2^i}_{\text{相当于}k\text{位全为1}} = 2^i(2^k - 1) = \underbrace{2^{i+k}}_{i+k\text{位上加1}} - \underbrace{2^i}_{i\text{位上减1}}$$

把 $(11)_2$ 表示成 $(11)_2 = (100)_2 - (01)_2$ 。即把当前的两位编码为“ $-(01)_2$ ”，同时需要在高一位增加1。因此，在给高2位编码时，需要考察低位的情况，来决定是否需要额外加1

改进的波兹编码

通过同时观察**3** 位乘数进一步简化

即：在乘数的低位上加一“考察位”：考察 $y_{i+1}y_iy_{i-1}$ ，其中 y_{i-1} 为“考察位”。下次考察时考察 $y_{i+3}y_{i+2}y_{i+1}$ ，其中 y_{i+1} 为“考察位”。相邻两次考察时重叠一位乘数（ y_{i+1} ）。

对于有符号数(**2's**补码表示)的乘数 Y

$$Y = -y_{n-1}2^{n-1} + \sum_{j=0}^{n-2} y_j 2^j$$
$$= \sum_{i=0}^{i=n/2-1} \underbrace{(y_{2i-1} + y_{2i} - 2y_{2i+1})}_{\{-2, -1, 0, 1, 2\}} 2^{2i}, y_{-1} = 0$$

编码公式

改进波兹编码的证明

$$X \bullet Y = (y_{n-1}2^{n-1} + y_{n-2}2^{n-2} + \cdots + y_12^1 + y_02^0) \bullet X$$

$$= (\overbrace{y_{-1} + y_0 - 2y_1}^{\equiv 0}) \bullet X \bullet 2^0 + \\ + (y_1 + y_2 - 2y_3) \bullet X \bullet 2^2 + \\ + (y_3 + y_4 - 2y_5) \bullet X \bullet 2^4 + \\ + \cdots +$$

$$+ (\overbrace{y_{n-1} + y_n - 2y_{n+1}}^{\equiv 0}) \bullet X \bullet 2^n$$

$$= (y_02^0 + y_12^1 + y_22^2 + y_32^3 + \cdots + y_{n-1}2^{n-1}) \bullet X \\ = X \bullet Y$$

$$= (\overbrace{y_{-1} + y_0 - 2y_1}^{\equiv 0}) \bullet X \bullet 2^0 + \quad (n \text{ 为奇数}) \\ + (y_1 + y_2 - 2y_3) \bullet X \bullet 2^2 + \\ + \cdots + \\ + (\overbrace{y_{n-2} + y_{n-1} - 2y_n}^{\equiv 0}) \bullet X \bullet 2^{n-1} = X \bullet Y$$

(若 n 为偶数)

改进波兹编码乘法

$y_{i+1}y_iy_{i-1}$	部分积
000	0
001	+X
010	+X
011	+2X
100	-2X
101	-X
110	-X
111	0

$$y_{2i-1} + y_{2i} - 2y_{2i+1}$$

举例：X = 1101, Y = 1001, 4 位乘 4 位 = 8 位

将 X、Y 看成是无符号的正数，则 X = 13, Y = 9, X · Y = 117

乘数 Y = 0 0 1 0 0 1 0

+X -2X +X

$$\begin{array}{r}
 00001101 \quad +X \\
 100110 \quad -2X \\
 1101 \quad +X \\
 \hline
 01110101 = 117
 \end{array}$$

注意：-2X 为左移，求反，加1（或求反，加1，左移）。左移时右边添0。

将 X、Y 看成2's补码（负数），则 X = -3, Y = -7, X · Y = 21

乘数 Y = 1 1 1 0 0 1 0

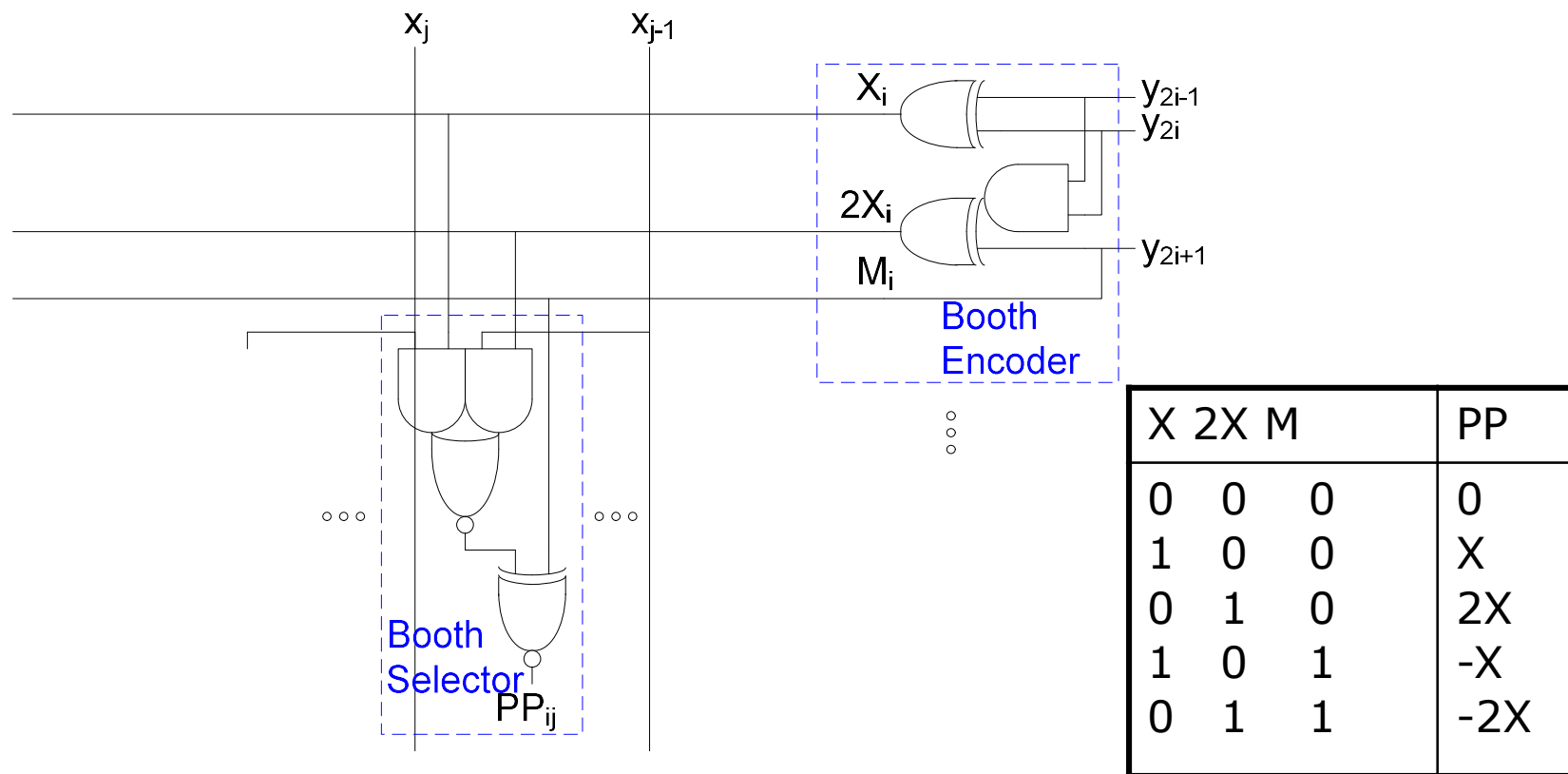
0 -2X +X

$$\begin{array}{r}
 1111101 \quad +X \\
 000110 \quad -2X \\
 \hline
 00010101 = 21
 \end{array}$$

计算与无符号数相同

Booth编码器及部分积选择器

波兹编码的部分积计算需要编码、译码、移位和（求反+1）



改进Booth编码乘法的特点

❖ 优缺点：

- 部分积数目减为原来的一半。可用于无符号数及2's 补码数。
- 代价是：部分积除0，X 外还有 $+2X$ ， $-2X$ ， $+X$ ， $-X$ ，因此需要译码。对负项还要“求反加1”

❖ 在部分积累加时每产生一个部分积要左移2位

❖ 也可以同时考查4位，每3位进行一次编码（基8波兹编码）

乘法器

❖ 二进制乘法器

❖ 部分积产生

❖ 部分积的累加

- 逐位进位阵列乘法器

- 进位保留乘法器

- 树形乘法器

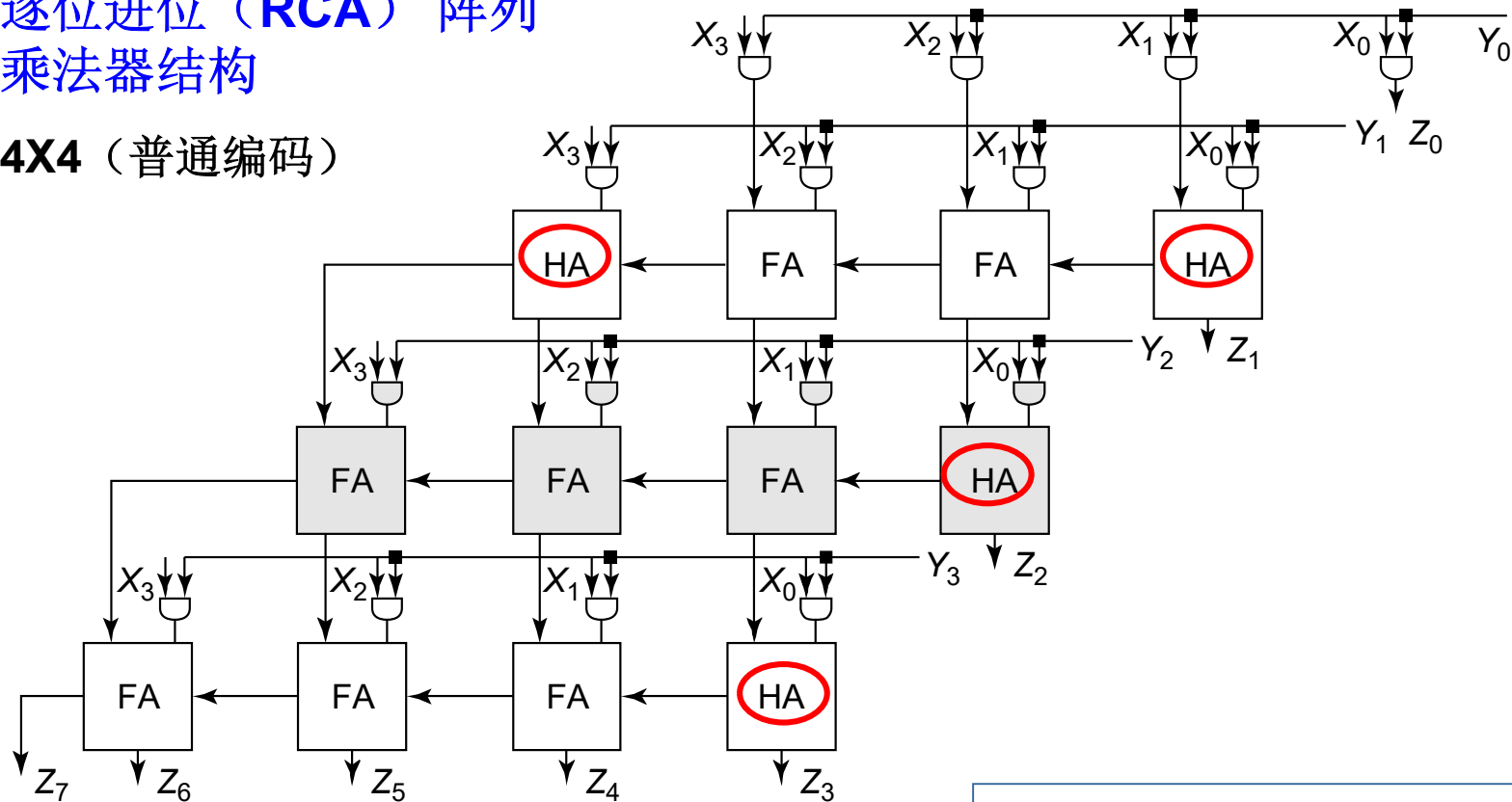
❖ 最终相加

❖ 高性能乘法器结构

阵列乘法器

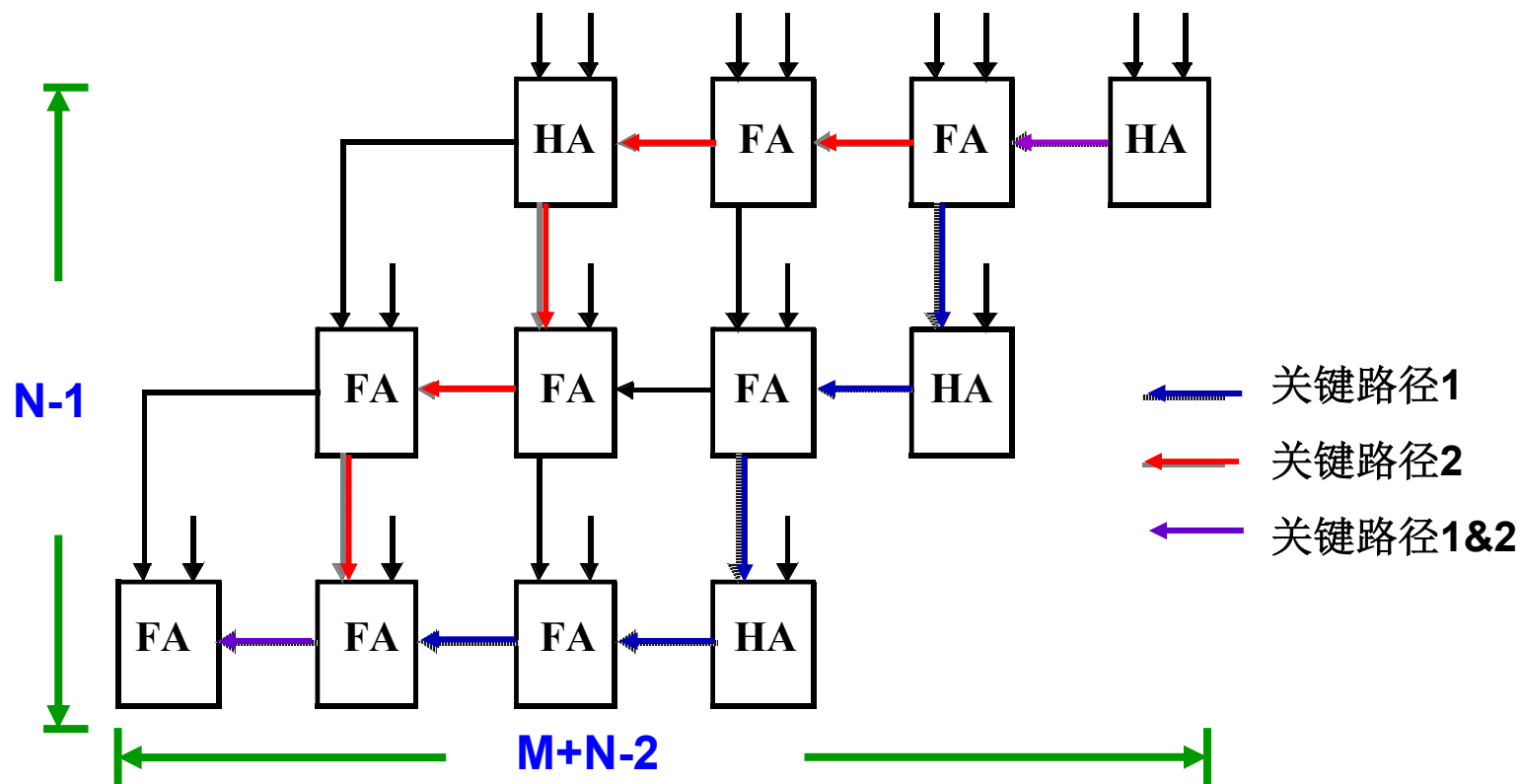
逐位进位（**RCA**）阵列
乘法器结构

4X4（普通编码）



$m \times n$ 位乘法器，共需：
 $n-1$ 个 M 位的加法器

MxN RCA阵列乘法器的关键路径



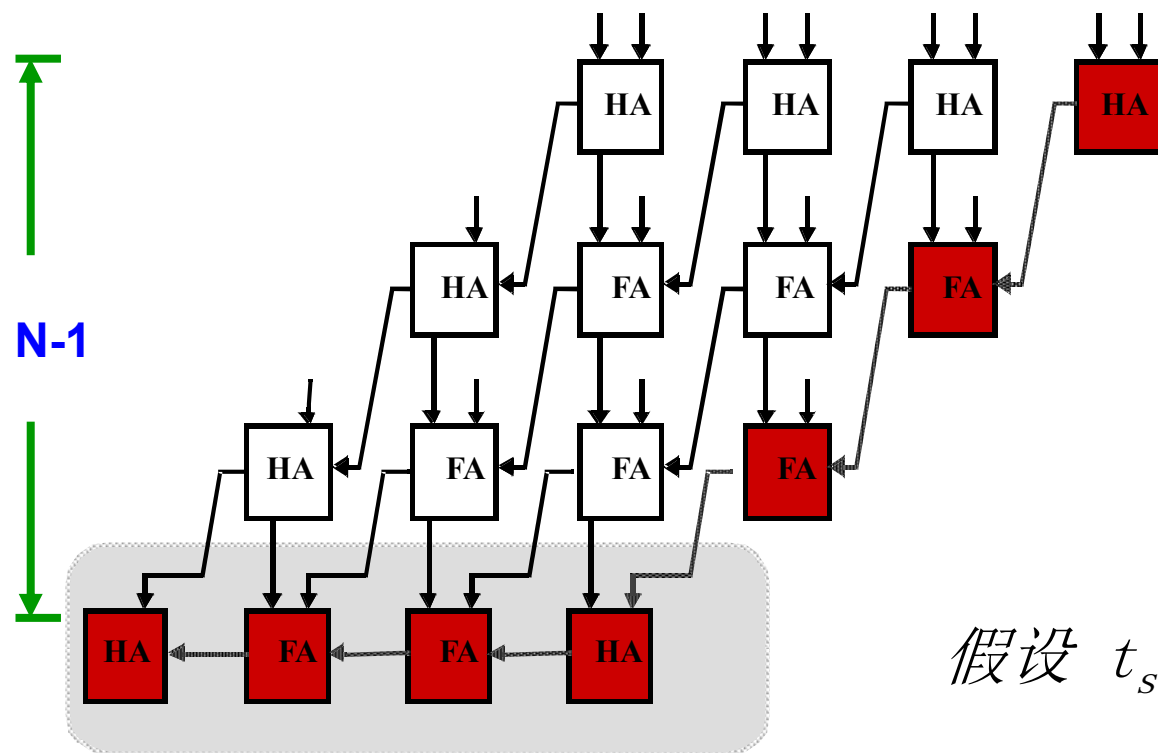
$$t_{mult} = t_{and} + [(M - 1) + (N - 2)]t_{carry} + (N - 1)t_{sum}$$

多条长度相同的关键路径； t_{sum} 和 t_{carry} 的延时同等重要

乘法器

- ❖ 二进制乘法器
- ❖ 部分积产生
- ❖ 部分积的累加
 - 逐位进位阵列乘法器
 - 进位保留乘法器
 - 树形乘法器
- ❖ 最终相加
- ❖ 高性能乘法器结构

进位保留乘法器结构



假设 $t_{sum} = t_{carry}$

向量合并加法器

$$t_{mult} = t_{and} + (N-1)t_{carry} + t_{merge}$$

进位保留加法器

- ✿ 一个 n 位的进位保留加法器（Carry-Save Adder, CSA）：
由 n 个不相关的一位全加器构成，在这些全加器之间没有进位关系，即在一个 n 位CSA加法器的内部没有Carry 传播。
- ✿ 它的输入是要被相加的三个 n 位数，它的输出是两个字：
 - 一个字是 n 位的“和”： $S(S_{n-1}, \dots, S_1, S_0)$
 - 另一个字是 n 位的“进位”： $C(C_{n-1}, \dots, C_1, C_0)$
- ✿ S 和 C 可以被送入另一个 n 位的CSA 与第四个 n 位的数 W 相加；一般地，多个 n 位的数可以通过CSA 树结构相加而产生。
- ✿ 为得到最后的和， S 和 C 必须最终用一个带进位传播的通常的加法器（CPA）相加（最终相加）。
- ✿ 在CSA中为了加快连续加法的速度，减少门延时，可以采用正码和反码两种CSA，在阵列中交叉使用。
- ✿ 进位保留乘法器在最坏情形下的关键路径是唯一确定的。

乘法器

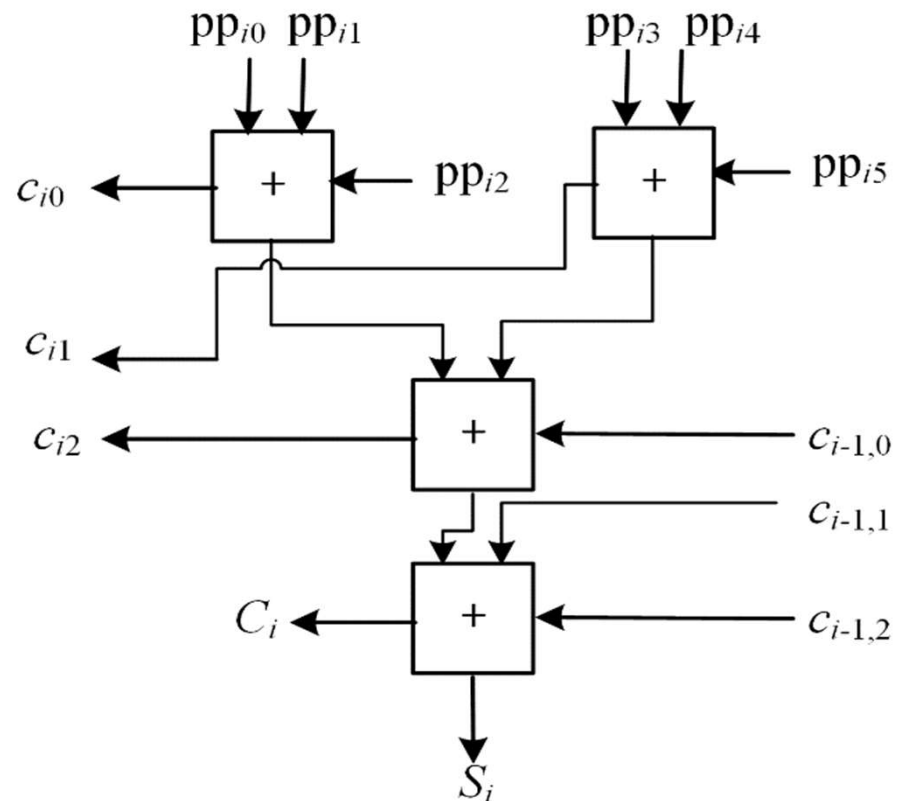
- ❖ 二进制乘法器
- ❖ 部分积产生
- ❖ 部分积的累加
 - 逐位进位阵列乘法器
 - 进位保留乘法器
 - 树形乘法器
- ❖ 最终相加
- ❖ 高性能乘法器结构

树形结构部分积累加

Wallace Tree 乘法器:

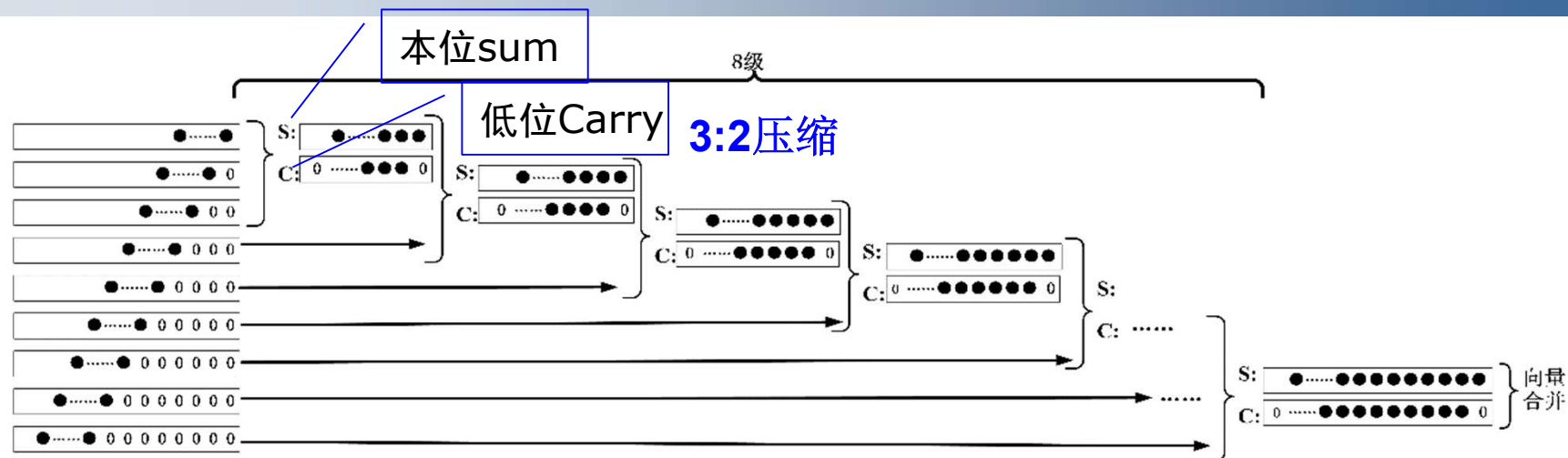
- 树形结构就是把同一列的多个加数（位）——包括右边一列的进位——分组累加后再把各组的和相加
- 分组累加由进位保留加法器或其它压缩电路实现，相当于把部分积进行了压缩。
- C 和 S 最终用进位传播加法器（CPA）相加。

pp_{i0} 表示第0个部分积的第*i*位, pp_{i1} 表示第1个部分积的第*i*位, c_{i0} 表示第0个进位向量的第*i*位输出.....

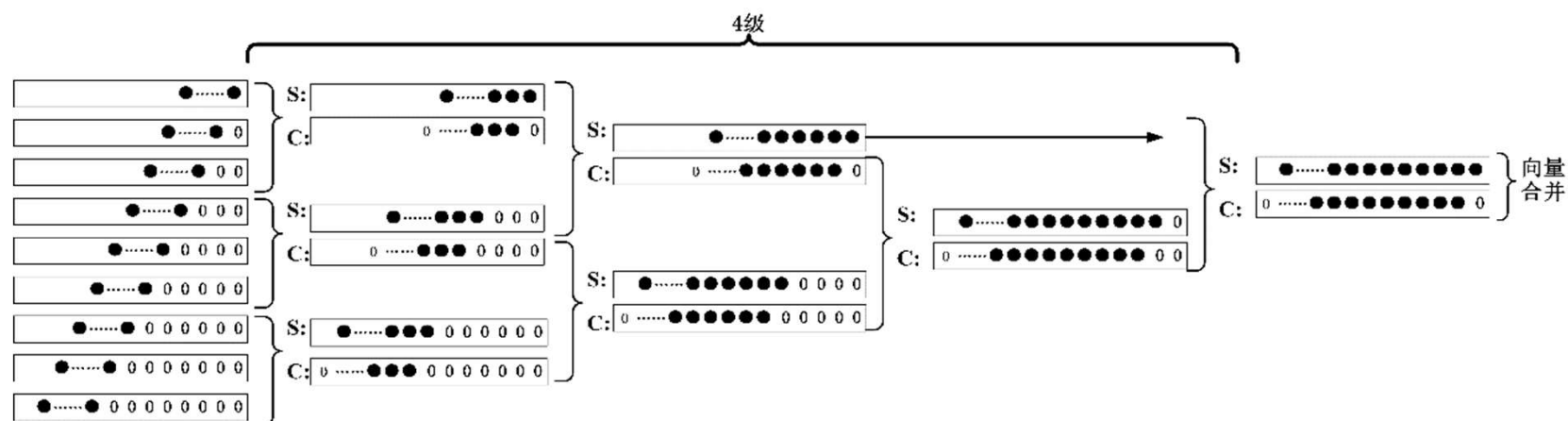


Wallace tree乘法器

Wallace Tree 乘法器与阵列乘法器的比较



(a) 线性进位保留乘法器



(b) 树形进位保留乘法器

Wallace Tree的特点

🌟 优点:

- 在通常的阵列乘法器中，乘法延迟时间正比于 n 。
- 采用压缩比为3:2的Wallace Tree 乘法器的乘法延迟时间正比于 $\log_{3/2} n$ 。最后还需将最终的两个数相加。

证明：有 $n \log_{3/2} \left(\frac{n}{2} \right)$ “部分积”，把 n 个“部分积”分成三个“部分积”为一组（共 $n/3$ 组）同时相加，每组相加的结果为两个数，共产生 $2(n/3)$ 个数。也就是 n 个向量（ n 个“部分积”）每加一次变为 $(2/3)n$ 个数，若经 x 次变为2个向量。

$$\text{则 } x = \log_{3/2} \left(\frac{n}{2} \right)$$

缺点：Wallace Tree 乘法器非常不规则，给版图设计带来困难

乘法器

- ❖ 二进制乘法器
- ❖ 部分积产生
- ❖ 部分积的累加
- ❖ 最终相加
- ❖ 高性能乘法器结构

最终相加

- ❖ 实现CSA或者Wallace Tree最后的向量合并
- ❖ 由进位传播加法器（CPA）实现，结构根据部分积累加阵列的结构而定
 - 如果所有输入同时到达(如流水线结构)，一般使用超前进位加法器
 - 输入到达时间不同，一般使用进位选择加法器，可以节省面积

乘法器

- ❖ 二进制乘法器

- ❖ 部分积产生

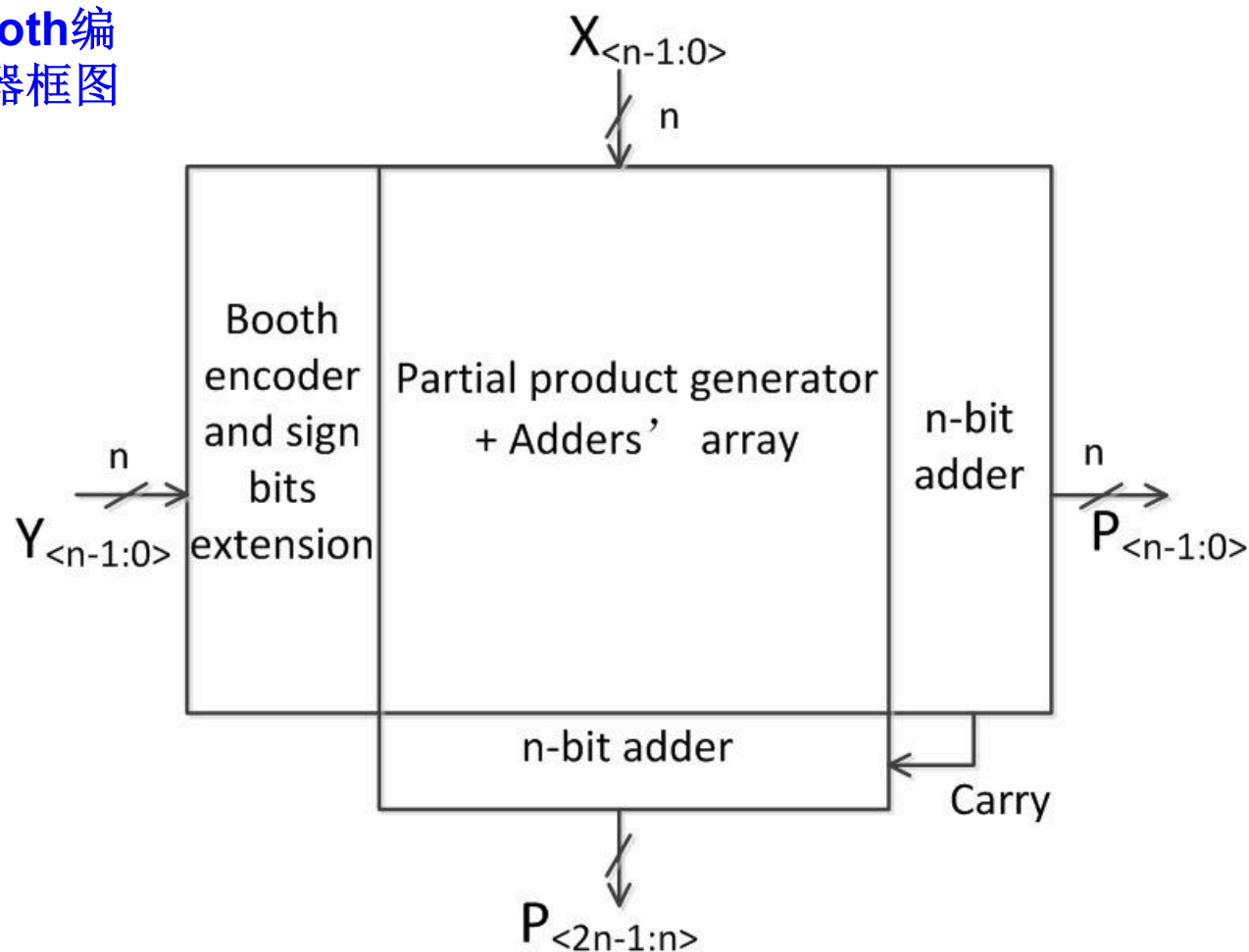
- ❖ 部分积的累加

- ❖ 最终相加

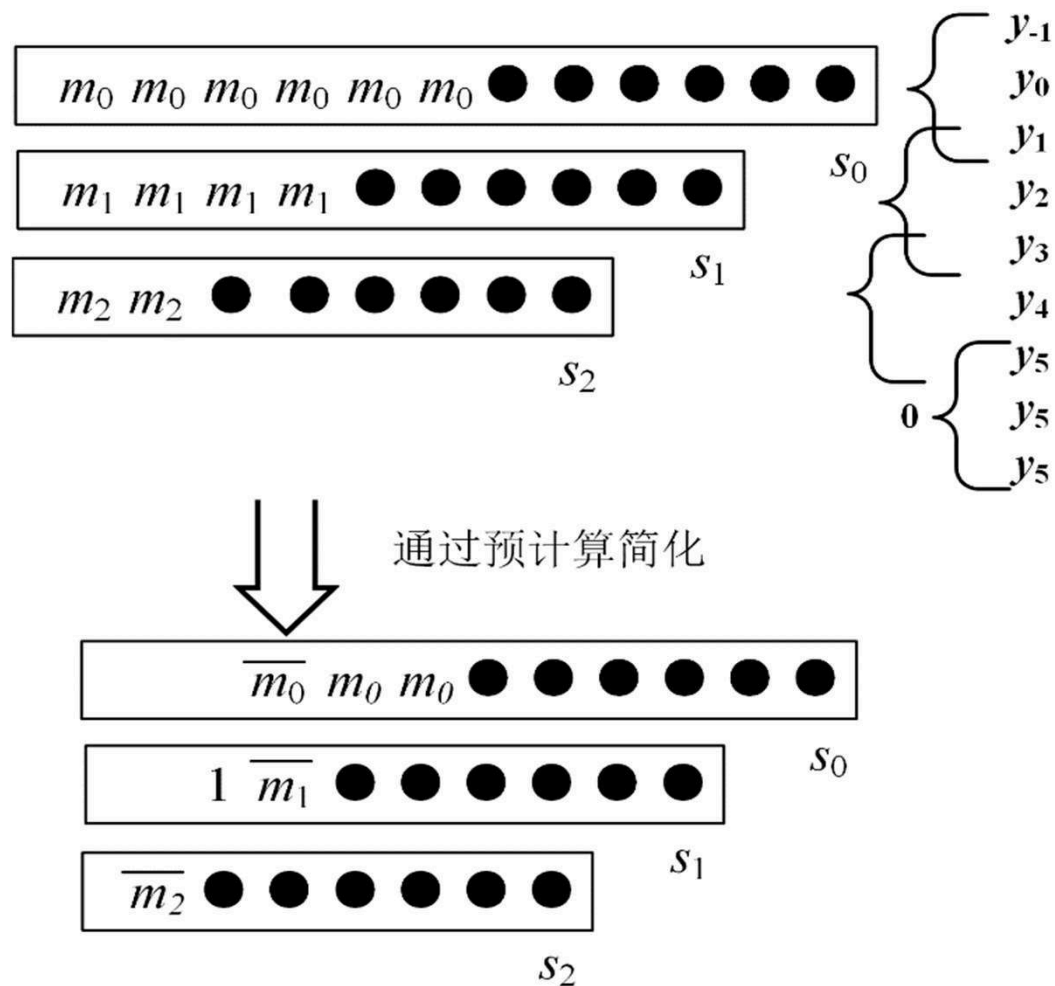
- ❖ 高性能乘法器结构

Booth编码CSA乘法器

改进Booth编
码乘法器框图



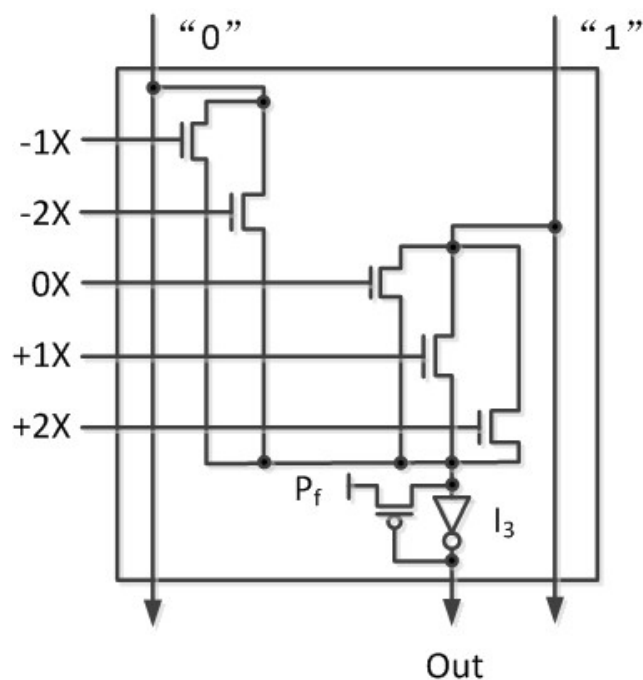
有符号乘法器的符号扩展化简



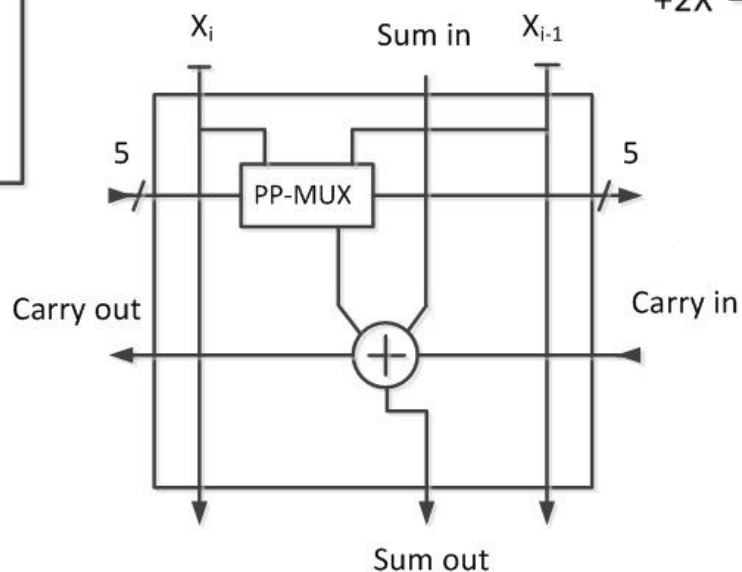
$$\begin{array}{r}
 1111\dots 1 \\
 + \quad \quad \overline{S} \\
 \hline
 SSSS\dots S
 \end{array}$$

6X6 Booth编码有符号数CSA乘法器

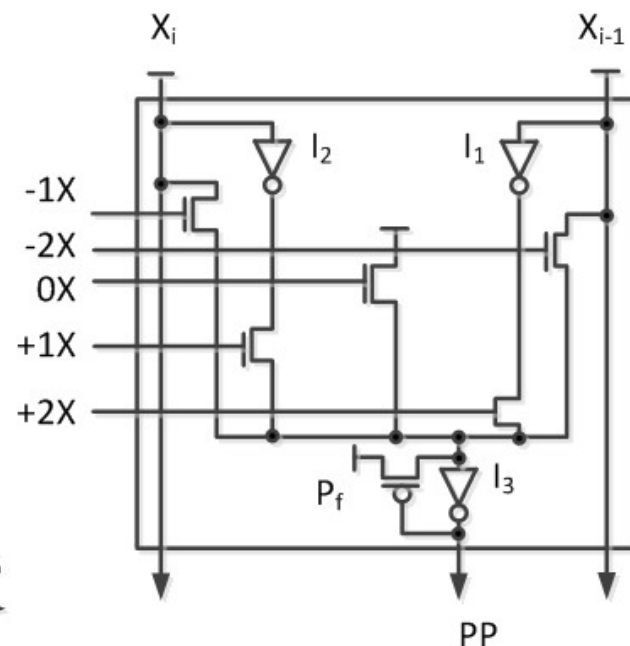
单元电路



ADD



PP-FA(PP-HA没有Carry_in)



PP-MUX（部分积选择器）

IPP-MUX是**PP-MUX**的反相输出

基本单元功能描述

- ❖ ADD: 产生每个部分积的末位, 如果编码为 $-2X$ 或 $-X$, 则输出1, 其输出在合并加法器中与累加结果相加
- ❖ PP-MUX: 按照编码结果选择部分积输出
- ❖ PP-FA (PP-HA): 包括两部分
 - PP-MUX
 - 加法器: 把前几级的S、本地的PP、上级的Carry_out相加, 产生S与Carry_out

乘法器小结

- ❖ 第一次系统级的优化
- ❖ 优化目标和二进制加法器不同
- ❖ 又一次出现**关键路径**的概念：识别关键路径至关重要
- ❖ 其它可能的优化技术
 - 对数结构-线性结构(高压缩比的Wallace Tree)
 - Booth编码（高基波兹编码）
 - 流水线技术

总结

❖ 数据通路的特点：

- 位片式
- 主要优化指标是性能和功耗，结构的规则性也非常重要

❖ 加法器：关键路径是进位传播路径

❖ 乘法器：

- 总延时=部分积时间+部分积累加时间+最终相加时间
- 可以通过波兹编码减少部分积数量

总结（续）



数据通路的优化方法

- 线性结构的旁路：进位旁路加法器、超前进位加法器、进位保留
- 先计算后选择：进位选择
- 分组并行处理：树形加法器、树形乘法器
- 流水线
- 把关键路径上的操作移到非关键路径上：利用加法器的反相特性，平方根进位选择……
- 预计算：在设计阶段完成常数运算，简化电路（乘法器的符号扩展位精简、波兹编码）